流水线 MIPS 处理器的设计报告

无 37 董皓彧

2026 年 1 月 9 日

1 实验目的

使用 Verilog 实现一个五级流水线 CPU，支持 MIPS 指令集的部分指令。在其上运行稀疏矩阵乘法算法，将结果显示在 BCD7 数码管上。

2 设计方案

2.1 支持的指令

CPU 支持如下指令

R 型指令	I 型指令	J 型指令
add	lw	j
addu	sw	jal
sub	lui	
subu	addi	
mul	addiu	
and	andi	
or	ori*	
xor	slti	
nor	sltiu	
sll	beq	
srl	bne	
sra	blez	
slt	bgtz	
sltu	bltz	
jr		
jalr		

* 非硬性要求实现的指令，但是 Mars 翻译出的机器码中存在该指令，需要支持。

表 1: CPU 支持的指令

2.2 流水线分级

CPU 设计采用五级流水线，分为取指 IF、译码 ID、执行 EX、存储器访问 MEM 和写回 WB 五个阶段，另设有不属于任何一个阶段的转发单元和冒险检测单元。各个阶段的功能如下：

- IF：从指令存储器中读取指令，并将程序计数器 PC 更新为下一条指令的地址。
- ID：译码指令，读取寄存器堆中的数据，并进行立即数扩展。
- EX：执行算术逻辑运算，计算跳转地址。
- MEM：访问数据存储器，读取或写入数据。
- WB：将数据写回寄存器堆。

2.3 模块设计

CPU 的设计分为以下几个模块：

- 顶层模块 top.
- CPU 模块 CPU.
- ALU 和 ALU 控制单元 ALU, ALUControlUnit.
- 寄存器堆 RegisterFile.
- 指令存储器 InstMem.
- 数据存储器 DataMem.
- 控制单元 ControlUnit.
- 程序计数器 PC.
- 转发单元 ForwardUnit.

- 冒险检测单元 HazardUnit.
- 立即数扩展单元 ImmExtendUnit.
- 级间寄存器 {IFID,IDEX,EXMEM, MEMWB}Regs.

2.4 模块框图

见图 1.

2.5 模块设计细节

2.5.1 顶层模块 top

顶层模块包括 CPU 和分频器。分频器使用锁相环 IP 实现，按照时序分析的结果使 CPU 在支持的最高时钟/频率下工作。

2.5.2 CPU 模块 CPU

CPU 模块将各个子模块连接起来，形成完整的 CPU。各个模块之间的简单逻辑也被放在该模块内实现，具体包括：

- 将 PC+4 的高 4 位与指令的立即数部分和 2'h00 拼接，形成完整的 j 指令的跳转地址。
- 将 PC+4 的高 4 位与指令的立即数部分和 2'h00 相加，形成完整的 branch 类型指令的跳转地址。
- 选择 PC 从何处转发 jr, jalr 指令所需的寄存器数据。
- 选择 ALUControlUnit 从 EXMEMRegs 转发时转发何值。
- 根据 RegDst 控制信号决定 EXMEMRegs.RegWrite_Addr 为何值。
- 选择 DataMem 从何处转发 DataMem.WriteData.

2.5.3 ALU 和 ALU 控制单元 ALU, ALUControlUnit

ALU 模块根据 ALUOp 控制信号进行相应的运算。ALU 控制单元根据 ALUSrc 控制信号和各个数据来源决定 ALU 的输入。

2.5.4 寄存器堆 RegisterFile

寄存器堆模块实现寄存器的读写操作。其中写入（包括复位）在时钟下降沿完成，以便 WB 阶段写回的数据可以立即供 ID 阶段的指令使用。

2.5.5 指令存储器 InstMem

存储指令的机器码。硬件实现时使用组合逻辑将指令硬编码到指令存储器中；仿真时通过读取参数 INST_FILE 指示的文件初始化指令存储器。具体见第 4.1 节。

2.5.6 数据存储器 DataMem

提供对 128 字节数据存储器的读写操作，以及映射为地址 0x40000010 的 BCD7 数码管的写操作。

2.5.7 控制单元 ControlUnit

指令的操作码生成各个控制信号。具体包括：

- PC 更新时的选择信号 PCSrc.
- PC 更新时判断是哪一种 branch 指令的信号 Branch_Type.
- 寄存器堆写回使能信号 RegWrite.
- 寄存器堆写回地址选择信号 RegDst.
- 寄存器堆写回数据的来源选择信号 MemtoReg.
- 数据存储器读取使能信号 MemRead.
- 数据存储器写入使能信号 MemWrite.
- ALU 输入 A、输入 B 的来源选择信号 ALUSrcA, ALUSrcB.

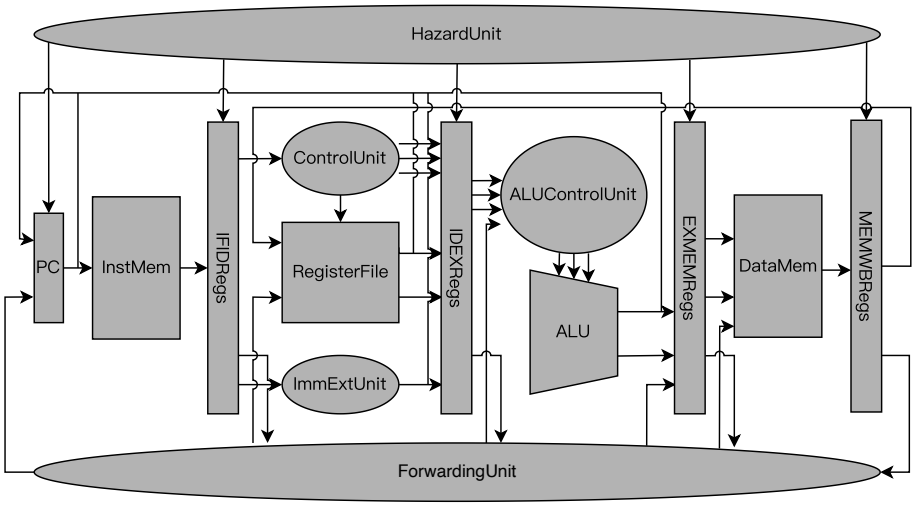


图 1: CPU 模块框图

- ALU 操作码 ALUOp.
- 立即数扩展操作码 ExtOp.

2.5.8 程序计数器 PC

存储当前指令的地址，并根据信号进行更新。

2.5.9 转发单元 ForwardUnit

解决能够通过数据转发解决的数据冒险。转发路径包括：

- 从 EXMEMRegs 和 MEMWBRegs 的输出转发到 ALUControlUnit 的输入。
- 从 MEMWBRegs 的输出转发到 DataMem 的输入，以解决 lw 后紧接 sw 对同一地址先读后写的数据冒险。
- 从 MEMWBRegs 的输出转发到 EXMEMRegs.RegRtData，以解决 lw 后隔一条指令然后 sw 对同一寄存器先读后写的数据冒险。[1]
- 从 EXMEMRegs 的输出转发到 PC 输入，以解决在 ID 阶段处理 jr 和 jalr 指令需要使用 $rs 寄存器中的值的数据冒险。

[1]这一条转发路径并不是一开始就注意到的。是经过第 4.1.2 节中的仿真后添加的。

2.5.10 冒险检测单元 HazardUnit

通过阻塞流水线来解决无法通过数据转发解决的冒险以及控制冒险，以及确定 PC 的 PCSrc 输入信号的值。冒险检测单元的逻辑包括：

- load-use 冒险：在 EX 阶段检测，阻塞 IF, ID 和 EX 阶段的指令一个周期，等待 lw 指令的结果写回寄存器堆。
- jr 或 jalr 指令需要使用前序第 1 或第 2 条指令的结果：在 ID 阶段检测，阻塞 IF 和 ID 阶段的指令一个周期，等待前序指令的结果写回寄存器堆。[2]
- 分支指令的控制冒险：在 EX 阶段检测，清除 IF 和 ID 阶段的指令。
- 跳转指令的控制冒险：在 ID 阶段检测，清除 IF 阶段的指令。

2.5.11 立即数扩展单元 ImmExtendUnit

根据 ExtOp 控制信号对指令的立即数进行符号扩展、零扩展或 lui 指令的高位填充。

[2]同上。

2.5.12 级间寄存器 {IFID,IDEX,EXMEM, MEMWB}Regs

各个阶段之间的级间寄存器。例如 IFIDRegs 即为 IF 阶段和 ID 阶段之间的寄存器，其余类似。用来存储后续阶段需要使用的数据和控制信号。

3 稀疏矩阵乘法的 MIPS 代码

稀疏矩阵乘法的 MIPS 代码主要分为两部分：稀疏矩阵乘法算法和 BCD7 数码管显示。由于本人选修的是秋季学期的数字逻辑与处理器课程，汇编大作业实现的不是稀疏矩阵乘法算法，故前一部分使用了已有的汇编代码。下面介绍后一部分的实现。

使用 BCD7 数码管对运算结果矩阵进行逐行逐元素的显示。对于每一个元素，利用视觉暂留效应，通过高频率刷新轮流显示每一位数字，实现人眼观察上各个数码管显示不同数字的效果。锁定 BCD7 数码管一次刷新的周期为 4096 个时钟周期，根据实际实现时的时钟频率调整显示每个矩阵元素时的刷新次数。由于结果矩阵的每个元素均在 16 位二进制数表示的范围内，且为了能够使一个元素一次性在 4 个 BCD7 数码管上显示，因此显示时仅显示了每个元素的低 16 位。

设计上 DataMem 的 BCD7 输出的 [7:0] 控制 {DP,CG,CF,...,CA} 的显示，[11:8] 控制数码管 {0,1,2,3} 的使能。数码管的显示结果与 BCD7[11:0] 的值的对应关系见表 2.

需要注意的是，显示十六进制数字时，字母部分全部按照大写显示。此时部分字母的显示样式会与部分数字相同，如 B 和 8、D 和 0 等。于是显示这两个字母时会同时将对应数码管的小数点点亮以示区别。这一点已经在表 2 中体现。

具体代码见 Listing 10.

数位†	BCD7[11:8]	数字	BCD7[7:0]
*___	0x1	0x0	0x3F
-*__	0x2	0x1	0x06
_*-_	0x4	0x2	0x5B
___*	0x8	0x3	0x4F
		0x4	0x66
		0x5	0x6D
		0x6	0x7D
		0x7	0x07
		0x8	0x7F
		0x9	0x6F
		0xA	0x77
		0xB	0xFF
		0xC	0x39
		0xD	0xBF
		0xE	0x79
		0xF	0x71

† "-" 表示该数码管不显示数字，"*" 表示该数码管显示数字。

表 2: BCD7 数码管显示结果与 BCD7[11:0] 的值的对应关系

4 行为级仿真

4.1 正确性验证

使用 iverilog 对 CPU 进行三次行为级仿真以验证设计的正确性。为方便仿真，通过 SIMULATION 宏控制 InstMem 条件编译。当进行仿真时，InstMem 通过 $readmemh 读取 Mars 转换对应汇编文件生成的机器码文件而非使用硬编码在 InstMem 中的指令进行初始化，从而可以在仅修改仿真代码而不修改 CPU 设计代码的情况下进行仿真。

4.1.1 算术指令和转发

本仿真（Listing 17, 16, 23）测试了表 1 中所有的算数指令[3]和大部分转发逻辑。在逐时钟周期

[3]在编写测试用例 1 时未考虑到 Mars 翻译出来的代码实际上包含了 ori 指令。而且理论上 ori 指令与 addi 指令除了 ALUOp 信号和 ALU 执行的算术操作以外并无不同，故后来也未对测试用例 1 进行

分析本仿真产生的波形文件的过程中，我纠正了一些线路连接上的错误，如文件 CPU.v 中的某些 wire 型变量未连接模块的输出端口，ControlUnit 的 ALUOp 输出位数错误等。

4.1.2 控制指令和冒险

本仿真（Listing 19, 18, 24）测试了表 1 中所有的分支指令和跳转指令以及冒险检测单元的功能。在逐时钟周期分析本仿真产生的波形文件的过程中，我补充了先前未注意到的一条转发路径和一个冒险检测逻辑：

1. 从 MEMWBRegs 级间寄存器的 RegWriteData 转发至 EXMEMRegs 级间寄存器的 RegRtData 以解决 lw 后隔一条指令然后 sw 对同一地址先读后写的数据冒险问题。

2. 若 IFIDRegs 级间寄存器的 PCSrc 指示本条指令需要从寄存器中读取跳转地址并跳转（如 jr 或 jalr 等），且 IFIDRegs 级间寄存器的 RegRsAddr 与 IDEXRegs 级间寄存器的 RegRtAddr 或 IDEXRegs 级间寄存器的 RegRdAddr 相同，或 EXMEMRegs 的 MemtoReg 指示写回寄存器堆的数据由存储器提供，则将 ID 阶段（及 ID 之前）的指令阻塞一个周期，等待数据准备完成。本条逻辑解决了 j 型指令在 ID 阶段便跳转引起的数据冒险，如 lw 后立即使用读取到的数据进行跳转的情况。

4.1.3 整体的稀疏矩阵乘法程序的测试

本仿真（Listing 21, 20, 25）测试了包括 BCD7 显示在内的整体的稀疏矩阵乘法程序。为方便仿真和观察，将 BCD7 数码管的刷新周期和显示每个矩阵元素时的刷新次数均减少至合适水平。由于经过了 4.1.1 节和 4.1.2 节的测试，在正确编码 BCD7 显示的内容后，通过观察 BCD7 的输出波形即简便地验证了算法和 CPU 设计的正确性。

补充。

4.1.4 仿真结果分析

以第 4.1.1 节中对算术指令和转发的仿真为例，通过对前若干个时钟周期内的仿真结果进行说明来展示大体的分析方法。如图 2：

1. 在第 5ns∼15ns 时钟周期内，PC 输出的地址为 0x00400000，对应汇编指令第一行 lui $s0, 0x00ff，其机器码为 0x3c1000ff，与该周期内 InstMem.Instruction 输出一致。

2. 在第 15ns∼25ns 时钟周期内，该指令处于 ID 阶段，其 $rs == 0, $rt = 0x1000，与 RegisterFile.RegRead_AddrA, RegisterFile.RegRead_AddrB 一致，同时寄存器堆正确地输出 0，即便这两个数据并不会在后续被使用。

3. 在第 25ns∼35ns 的时钟周期内，该指令处于 EX 阶段，对应 ALU 操作为加法，输入为扩展后的立即数，正确地对应 ALU.InA == 0, ALU.InB == 0x00ff0000，且 ALU.ALUOut == 0x00ff0000，输出正确。

4. 在第 35ns∼45ns 的时钟周期内，该指令处于 MEM 阶段。由于 lui 指令不涉及存储器操作，DataMem.MemRead == 0, DataMem.MemWrite == 0，但存储器仍会解析 $rt。波形图与理论相符。

5. 在第 45ns∼55ns 的时钟周期内，该指令处于 WB 阶段。从波形图看出，其 RegisterFile.RegWrite == 1，CPU 确实将 0x00ff0000 写入了 0x10000 号寄存器。

类似地，通过逐指令地分析整个波形文件，并对每条指令逐周期地分析，可以确认 CPU 这部分的功能正确，因此可以推测其他未列出的信号也大概率正确。

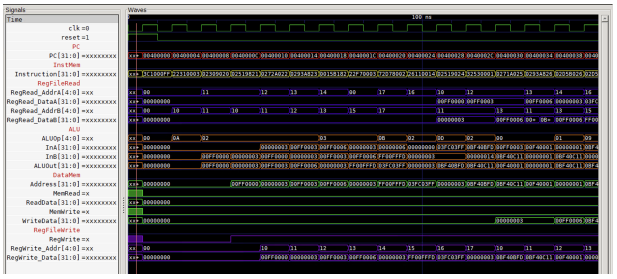
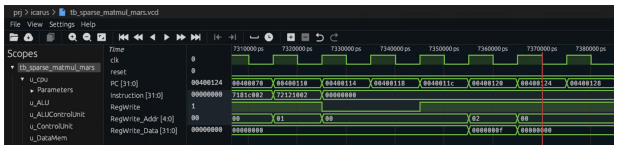


图 2: 算术指令和转发仿真结果分析示例



(a) 起始时间



(b) 结束时间

图 4: 稀疏矩阵乘法算法行为级仿真波形图

4.2 测试稀疏矩阵乘法算法的 CPI

将完整的稀疏矩阵乘法汇编程序中的稀疏矩阵乘法部分代码截取出来（Listing 14），通过 Mars 仿真器统计指令数为 653 条（图 3）。需要注意的是 Mars 仿真器默认数据存储地址与 Verilog 实现的数据存储器不同，需要在 Memory Configuration 中选择 Compact, Data at Address 0 选项。

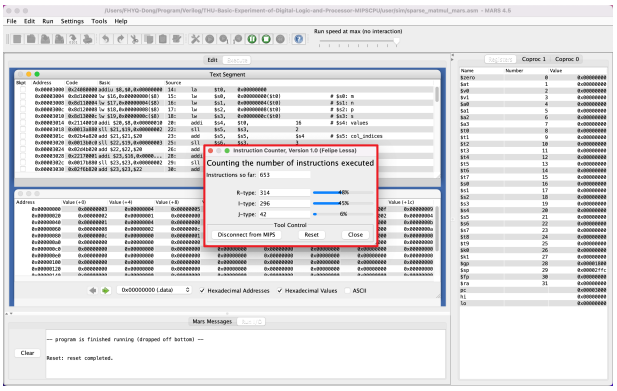


图 3: Mars 仿真器统计指令数

同时，使用 Mars 将该汇编代码的.text 部分转换为机器码，进行行为级仿真，从波形文件中读出程序起始于 10000ps，结束于 7370000ps，时钟周期为 10000ps（图 4）。执行这段程序消耗了 736 个时钟周期，因此稀疏矩阵乘法算法的 CPI 为

$$\mathrm{CPI} = \frac{\text{时钟周期数}}{\text{指令数}} = \frac{736}{653} \approx 1.13 \qquad (1)$$

5 综合

5.1 时序分析和时钟频率

设置时钟周期为 10ns，占空比为 50%，在 Vivado 中综合后进行时序分析。结果显示建立时间约束违例 -5.827ns，保持时间约束富裕 0.127ns，关键路径见图 5. 需要注意的是 Vivado 可能会将一些逻辑比较简单的模块合并到其他模块中，需要在实例化模块时使用 (* keep_hierarchy = “yes” *) 语句来保留模块层次结构以便更清晰地观察关键路径。

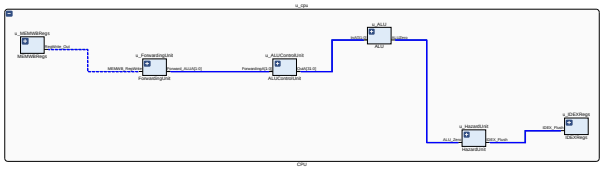


图 5: 时序分析建立时间的关键路径

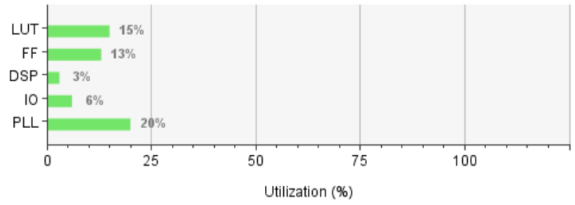
将时钟周期改为 16ns，占空比为 50%，重新进行时序分析，结果显示建立时间和保持时间约束均满足。于是使用锁相环 IP 将原始时钟信号分频至 60MHz，作为 CPU 的时钟。

5.2 资源占用

资源占用情况如图 6. 各资源占用情况均不超过 20%. 其中，LUT 主要在 DataMem 和 ALU 中被使用，主要负责逻辑运算；FF 主要在 DataMem 和 RegisterFile 中被使用，主要负责存储；MUX

主要在 DataMem 和 RegisterFile 中被使用，主要负责选择存储单元；DSP 仅在 ALU 中被使用，负责乘法运算；PLL 用于分频器，负责将原始时钟信号分频至 60MHz。

Resource	Utilization	Available	Utilization %
LUT	3171	20800	15.25
FF	5514	41600	13.25
DSP	3	90	3.33
IO	14	250	5.60
PLL	1	5	20.00



(a) 总体情况

Name	Slice LUTs (20800)	Slice Registers (41600)	F7 Muxes (16300)	F8 Muxes (8150)	Slice (8150)	LUT as Logic (20800)	LUT Flip Flop Pairs (20800)	DSPs (90)	Bonded IOB (250)	BUFGCTRL (32)	PLLE2_ADV (5)
top	3171	5514	805	256	2399	3171	268	3	14	2	1
clkwiz_inst (clk_wiz_0)	0	0	0	0	0	0	0	0	0	2	1
inst (clk_wiz_0_clk_...	0	0	0	0	0	0	0	0	0	2	1
u_cpu (CPU)	3171	5514	805	256	2399	3171	268	3	0	0	0
u_ALU (ALU)	458	0	0	0	141	458	0	3	0	0	0
u_ALUControlUnit (...	128	0	0	0	55	128	0	0	0	0	0
u_ControlUnit (Cont...	42	0	0	0	20	42	0	0	0	0	0
u_DataMem (DataM...	1307	4108	512	256	1749	1307	0	0	0	0	0
u_EXMEMRegs (EX...	0	111	0	0	52	0	0	0	0	0	0
u_ForwardingUnit (...	22	0	0	0	9	22	0	0	0	0	0
u_HazardUnit (Haza...	17	0	0	0	9	17	0	0	0	0	0
u_IDEXRegs (IDEX...	1	169	0	0	38	1	0	0	0	0	0
u_IFIDRegs (IFIDR...	1	64	0	0	29	1	0	0	0	0	0
u_ImmExtendUnit (I...	17	0	0	0	10	17	0	0	0	0	0
u_InstMem (InstMem)	219	0	37	0	66	219	0	0	0	0	0
u_MEMWBRegs (M...	32	38	0	0	24	32	32	0	0	0	0
u_PC (PC)	33	32	0	0	20	33	32	0	0	0	0
u_RegisterFile (Re...	730	992	256	0	488	730	61	0	0	0	0

(b) 各模块占用情况

图 6: 资源占用情况

6 硬件实现

由于经过了前期完善的仿真测试，在时钟频率调整合适、综合完成、确定硬件资源占用合理后，将生成的比特流直接烧录到 FPGA 上即可按照预期运行。对应的比特流文件位置见第 A.1 节中的项目文件结构。

7 感想体会

7.1 开源工作流的便捷性

在本实验中，除了最后 Implementation 阶段必须使用 Vivado 外，在编写 Verilog 代码和行为级仿真过程中均使用了开源工具如 iverilog[4]和 GTKWave[5]。这些工具相比 Vivado 而言较为轻量化、运行速度较快，且可以方便地通过命令行进行操作，极大地提高了实验效率。此外，开源工具的使用也使得代码的可移植性更强，可以在不同的操作系统上运行。

7.2 仿真测试的重要性

在本实验中，我通过行为级仿真全面地测试了 CPU 的功能的正确性，并成功地发现了多个潜在的 bug。正是因为在仿真阶段进行了充分的测试，才使得后续阶段能够迅速顺利地完成。

[4]https://github.com/steveicarus/iverilog
[5]https://github.com/gtkwave/gtkwave

A 源代码

A.1 项目文件结构

项目文件结构如下。使用 iverilog 进行仿真时请将工作目录切换到 user/的父目录下。

```
user/
├── src/ # 存放各CPU设计代码
│   ├── top.v                       # 顶层模块，包括CPU和分频器
│   ├── ALU.v                       # ALU和ALU控制单元的实现
│   ├── Control.v                   # 控制单元的实现
│   ├── CPU.v                       # 将各个模块连接成为一个完整的CPU
│   ├── Forwarding.v                # 转发单元的实现
│   ├── Hazard.v                    # 冒险检测单元的实现
│   ├── Memory.v                    # 指令存储器和数据存储器的实现
│   ├── PC.v                        # PC寄存器和相关逻辑的实现
│   ├── Register.v                  # 寄存器堆、级间寄存器的实现
│   ├── sparse_matmul.asm           # 稀疏矩阵乘法的汇编代码
│   └── sparse_matmul.inst          # 由Mars将汇编代码翻译而成的机器码
├── sim/ # 存放仿真相关代码
│   ├── asm2inst.py                 # 使用python调用Mars将汇编代码转换为机器码
│   ├── Mars4_5.jar                 # Mars仿真器的jar包
│   ├── printbin.py                 # 使用python将二进制文件中的数据打印出来
│   ├── tb_sparse_matmul_alog.asm   # 仅含有稀疏矩阵乘法算法的汇编代码，用于统计指令数
│   ├── tb_sparse_matmul_alog.inst  # 由Mars将稀疏矩阵乘法算法的汇编代码翻译成的机器码
│   ├── tb_sparse_matmul_alog.v     # 稀疏矩阵乘法算法测试模块的Verilog代码
│   ├── tb_sparse_matmul_alog.in    # 二进制文件，稀疏矩阵乘法算法的输入数据
│   ├── tb_sparse_matmul_alog.out   # 二进制文件，稀疏矩阵乘法算法的输出数据
│   ├── tb_CPU_inst_1.asm           # CPU测试用例1的汇编代码，包括算术指令和转发
│   ├── tb_CPU_inst_1.inst          # 由Mars将CPU测试用例1的汇编代码翻译成的机器码
│   ├── tb_CPU_inst_1.v             # CPU测试用例1的Verilog代码
│   ├── tb_CPU_inst_2.asm           # CPU测试用例2的汇编代码，包括控制指令和冒险
│   ├── tb_CPU_inst_2.inst          # 由Mars将CPU测试用例2的汇编代码翻译成的机器码
│   ├── tb_CPU_inst_2.v             # CPU测试用例2的Verilog代码
│   ├── tb_CPU_inst_3.asm           # CPU测试用例3的汇编代码，包括稀疏矩阵乘法和BCD7显示
│   ├── tb_CPU_inst_3.inst          # 由Mars将CPU测试用例3的汇编代码翻译成的机器码
│   └── tb_CPU_inst_3.v             # CPU测试用例3的Verilog代码
├── data/
│   ├── MIPSCPU.xdc                 # Vivado的约束文件
│   └── bin/
│       └── top.bin                 # Vivado生成的bitstream
└── ip/ # Vivado生成的ip核
```

A.2 硬件设计代码

Listing 1: user/src/top.v

```
`define SPARSE_MATMUL

module top (
    input wire             clk_in,
    input wire             reset,
    output wire [12 -1: 0] BCD7
);
    // After STA, the clock is set to 60MHz.
    wire clk_div, div_locked;
    clk_wiz_0 clkwiz_inst (
        .clk_in1  (clk_in),
        .reset    (reset),
        .clk_out1 (clk_div),
        .locked   (div_locked)
    );
    CPU u_cpu (
        .clk   (clk_div),
        .reset (reset || ~div_locked),
        .BCD7  (BCD7)
    );

    // // STA
    // CPU u_cpu (
    //     .clk   (clk_in),
    //     .reset (reset),
    //     .BCD7  (BCD7)
    // );
endmodule
```

Listing 2: user/src/ALU.v

```
module ALUControlUnit (
    input wire  [ 2 -1: 0] ALUSrcA,
    input wire  [ 2 -1: 0] ALUSrcB,
    input wire  [32 -1: 0] IDEX_RegRsData,
    input wire  [32 -1: 0] IDEX_RegRtData,
    input wire  [ 5 -1: 0] IDEX_Shamt,
    input wire  [32 -1: 0] IDEX_ImmExtend,
```

```
    input wire  [32 -1: 0] Forward_EXMEMRegData,
    input wire  [32 -1: 0] Forward_MEMWBRegData,
    input wire  [ 2 -1: 0] ForwardingA,
    input wire  [ 2 -1: 0] ForwardingB,

    output wire [32 -1: 0] OutA,
    output wire [32 -1: 0] OutB
);
    // forwarding flags
    parameter Forwarding_NONE  = 2'b00;
    parameter Forwarding_EXMEM = 2'b01;
    parameter Forwarding_MEMWB = 2'b10;
    // ALUSrc
    parameter ALUSrc_Reg   = 2'b11; // ALUSrcA: Rs, ALUSrcB: Rt
    parameter ALUSrc_Shamt = 2'b01;
    parameter ALUSrc_Imm   = 2'b10;
    parameter ALUSrc_None  = 2'b00;

    assign OutA = (ALUSrcA == ALUSrc_Reg)   ?
                      (ForwardingA == Forwarding_EXMEM ? Forward_EXMEMRegData :
                       ForwardingA == Forwarding_MEMWB ? Forward_MEMWBRegData :
                       IDEX_RegRsData) :
                  (ALUSrcA == ALUSrc_Shamt) ? {27'h000_0000, IDEX_Shamt} :
                  (ALUSrcA == ALUSrc_Imm)   ? IDEX_ImmExtend :
                  (ALUSrcA == ALUSrc_None)  ? 32'h0000_0000 :
                   32'h0000_0000;
    assign OutB = (ALUSrcB == ALUSrc_Reg)   ?
                      (ForwardingB == Forwarding_EXMEM ? Forward_EXMEMRegData :
                       ForwardingB == Forwarding_MEMWB ? Forward_MEMWBRegData :
                       IDEX_RegRtData) :
                  (ALUSrcB == ALUSrc_Shamt) ? {27'h000_0000, IDEX_Shamt} :
                  (ALUSrcB == ALUSrc_Imm)   ? IDEX_ImmExtend :
                  (ALUSrcB == ALUSrc_None)  ? 32'h0000_0000 :
                   32'h0000_0000;
endmodule


module ALU (
    input wire  [32 -1: 0] InA,
    input wire  [32 -1: 0] InB,
    input wire  [ 5 -1: 0] ALUOp,
```

```
49
50      output wire [32 -1: 0] ALUOut,
51      output wire            ALUZero
52  );
53      // ALUOp
54      parameter ALUOp_NOP  = 5'b1_1111; // No Operation
55      parameter ALUOp_AND  = 5'b0_0000; // And
56      parameter ALUOp_OR   = 5'b0_0001; // Or
57      parameter ALUOp_ADD  = 5'b0_0010; // Add
58      parameter ALUOp_SUB  = 5'b0_0011; // Substract
59      parameter ALUOp_SLT  = 5'b0_0100; // Set Less Than
60      parameter ALUOp_SLTU = 5'b0_0101; // Set Less Than Unsigned
61      parameter ALUOp_NOR  = 5'b0_1000; // Nor
62      parameter ALUOp_XOR  = 5'b0_1001; // Xor
63      parameter ALUOp_SLL  = 5'b0_1010; // Shift Left Logical
64      parameter ALUOp_SRL  = 5'b0_1011; // Shift Right Logical
65      parameter ALUOp_SRA  = 5'b0_1100; // Shift Right Arithmetic
66      parameter ALUOp_MUL  = 5'b0_1101; // Multiply
67
68      wire [32 -1: 0] ALUOut_Wire = (ALUOp == ALUOp_NOP)  ? 32'h0000_0000 :
69                                    (ALUOp == ALUOp_OR)   ? (InA | InB) :
70                                    (ALUOp == ALUOp_AND)  ? (InA & InB) :
71                                    (ALUOp == ALUOp_ADD)  ? (InA + InB) :
72                                    (ALUOp == ALUOp_SUB)  ? (InA - InB) :
73                                    (ALUOp == ALUOp_SLT)  ?
74                                        {31'h0000_0000, ((InA[31] ^ InB[31]) ?
75                                            (({InA[31], InB[31]} == 2'b01) ? 1'b0 : 1'b1) :
76                                            ((InA[30: 0] < InB[30: 0]) ? 1'b1 : 1'b0))} :
77                                    (ALUOp == ALUOp_SLTU) ? {31'h0000_0000, ((InA < InB) ? 1'
    b1 : 1'b0)} :
78                                    (ALUOp == ALUOp_NOR)  ? ~(InA | InB) :
79                                    (ALUOp == ALUOp_XOR)  ? (InA ^ InB) :
80                                    (ALUOp == ALUOp_SLL)  ? (InB << InA[4: 0]) :
81                                    (ALUOp == ALUOp_SRL)  ? (InB >> InA[4: 0]) :
82                                    (ALUOp == ALUOp_SRA)  ? ({{32{InB[31]}}, InB} >> InA[4:
    0]) :
83                                    (ALUOp == ALUOp_MUL)  ? (InA * InB) :
84                                    32'h0000_0000;
85      assign ALUOut = ALUOut_Wire;
86      assign ALUZero = (ALUOut_Wire == 32'h0000_0000) ? 1'b1 : 1'b0;
87  endmodule
```

```
88
89
90  module ImmExtendUnit (
91      input wire  [16 -1: 0] Imm_In,
92      input wire  [ 2 -1: 0] ExtOp,
93
94      output wire [32 -1: 0] Imm_Out
95  );
96      // ExtOp
97      parameter ExtOp_SignExtend = 2'b11;
98      parameter ExtOp_ZeroExtend = 2'b01;
99      parameter ExtOp_LUIExtend  = 2'b10;
100     parameter ExtOp_None       = 2'b00;
101
102     assign Imm_Out = (ExtOp == ExtOp_SignExtend) ? {{16{Imm_In[15]}}, Imm_In} :
103                      (ExtOp == ExtOp_ZeroExtend) ? {16'h0000, Imm_In} :
104                      (ExtOp == ExtOp_LUIExtend)  ? {Imm_In, 16'h0000} :
105                      (ExtOp == ExtOp_None)       ? 32'h0000_0000 :
106                      32'h0000_0000;
107 endmodule
```

Listing 3: user/src/Control.v

```
1   module ControlUnit (
2       input wire  [ 6 -1: 0] OpCode,
3       input wire  [ 6 -1: 0] Funct,
4
5       // Output control signals
6       output wire [ 2 -1: 0] PCSrc,
7       output wire [ 3 -1: 0] Branch_Type,
8       output wire            RegWrite,
9       output wire [ 2 -1: 0] RegDst,
10      output wire            MemRead,
11      output wire            MemWrite,
12      output wire [ 2 -1: 0] MemtoReg,
13      output wire [ 2 -1: 0] ALUSrcA,
14      output wire [ 2 -1: 0] ALUSrcB,
15      output wire [ 2 -1: 0] ExtOp,
16      output wire [ 5 -1: 0] ALUOp
17  );
18      // OpCode
```

```
19      parameter OpCode_RType = 6'h00;
20      parameter OpCode_LW    = 6'h23;
21      parameter OpCode_SW    = 6'h2b;
22      parameter OpCode_LUI   = 6'h0f;
23      parameter OpCode_ADD   = 6'h00;
24      parameter OpCode_ADDU  = 6'h00;
25      parameter OpCode_SUB   = 6'h00;
26      parameter OpCode_SUBU  = 6'h00;
27      parameter OpCode_MUL   = 6'h1c;
28      parameter OpCode_ADDI  = 6'h08;
29      parameter OpCode_ADDIU = 6'h09;
30      parameter OpCode_AND   = 6'h00;
31      parameter OpCode_OR    = 6'h00;
32      parameter OpCode_XOR   = 6'h00;
33      parameter OpCode_NOR   = 6'h00;
34      parameter OpCode_ANDI  = 6'h0c;
35      parameter OpCode_ORI   = 6'h0d;
36      parameter OpCode_SLL   = 6'h00;
37      parameter OpCode_SRL   = 6'h00;
38      parameter OpCode_SRA   = 6'h00;
39      parameter OpCode_SLT   = 6'h00;
40      parameter OpCode_SLTU  = 6'h00;
41      parameter OpCode_SLTI  = 6'h0a;
42      parameter OpCode_SLTIU = 6'h0b;
43      parameter OpCode_BEQ   = 6'h04;
44      parameter OpCode_BNE   = 6'h05;
45      parameter OpCode_BLEZ  = 6'h06;
46      parameter OpCode_BGTZ  = 6'h07;
47      parameter OpCode_BLTZ  = 6'h01;
48      parameter OpCode_J     = 6'h02;
49      parameter OpCode_JAL   = 6'h03;
50      parameter OpCode_JR    = 6'h00;
51      parameter OpCode_JALR  = 6'h00;
52      // Funct
53      parameter Funct_ADD  = 6'h20;
54      parameter Funct_ADDU = 6'h21;
55      parameter Funct_SUB  = 6'h22;
56      parameter Funct_SUBU = 6'h23;
57      parameter Funct_AND  = 6'h24;
58      parameter Funct_OR   = 6'h25;
59      parameter Funct_XOR  = 6'h26;
```

```
60      parameter Funct_NOR  = 6'h27;
61      parameter Funct_SLL  = 6'h00;
62      parameter Funct_SRL  = 6'h02;
63      parameter Funct_SRA  = 6'h03;
64      parameter Funct_SLT  = 6'h2a;
65      parameter Funct_SLTU = 6'h2b;
66      parameter Funct_JR   = 6'h08;
67      parameter Funct_JALR = 6'h09;
68      // PCSrc
69      parameter PCSrc_Branch  = 2'b11;
70      parameter PCSrc_Jump    = 2'b01;
71      parameter PCSrc_JumpR   = 2'b10;
72      parameter PCSrc_PCPlus4 = 2'b00;
73      // Branch Type
74      parameter Branch_Type_NONE = 3'b000;
75      parameter Branch_Type_BEQ  = 3'b101;
76      parameter Branch_Type_BNE  = 3'b001;
77      parameter Branch_Type_BLEZ = 3'b010;
78      parameter Branch_Type_BGTZ = 3'b011;
79      parameter Branch_Type_BLTZ = 3'b100;
80      // RegDst
81      parameter RegDst_RegRtAddr = 2'b11;
82      parameter RegDst_RegRdAddr = 2'b01;
83      parameter RegDst_RegRaAddr = 2'b10;
84      parameter RegDst_RegNone   = 2'b00;
85      // MemtoReg
86      parameter MemtoReg_MemData = 2'b11;
87      parameter MemtoReg_PCPlus4 = 2'b01;
88      parameter MemtoReg_ALUOut  = 2'b10;
89      parameter MemtoReg_None    = 2'b00;
90      // ALUSrc
91      parameter ALUSrc_Reg   = 2'b11; // ALUSrcA: Rs, ALUSrcB: Rt
92      parameter ALUSrc_Shamt = 2'b01;
93      parameter ALUSrc_Imm   = 2'b10;
94      parameter ALUSrc_None  = 2'b00;
95      // ExtOp
96      parameter ExtOp_SignExtend = 2'b11;
97      parameter ExtOp_ZeroExtend = 2'b01;
98      parameter ExtOp_LUIExtend  = 2'b10;
99      parameter ExtOp_None       = 2'b00;
100     // ALUOp
```

```
101     parameter ALUOp_NOP  = 5'b1_1111; // No Operation
102     parameter ALUOp_AND  = 5'b0_0000; // And
103     parameter ALUOp_OR   = 5'b0_0001; // Or
104     parameter ALUOp_ADD  = 5'b0_0010; // Add
105     parameter ALUOp_SUB  = 5'b0_0011; // Substract
106     parameter ALUOp_SLT  = 5'b0_0100; // Set Less Than
107     parameter ALUOp_SLTU = 5'b0_0101; // Set Less Than Unsigned
108     parameter ALUOp_NOR  = 5'b0_1000; // Nor
109     parameter ALUOp_XOR  = 5'b0_1001; // Xor
110     parameter ALUOp_SLL  = 5'b0_1010; // Shift Left Logical
111     parameter ALUOp_SRL  = 5'b0_1011; // Shift Right Logical
112     parameter ALUOp_SRA  = 5'b0_1100; // Shift Right Arithmetic
113     parameter ALUOp_MUL  = 5'b0_1101; // Multiply
114
115     // PCSrc: if branch, then branch; if jump, then jump; if jump register, then jump
    register; else PC + 4
116     assign PCSrc =
117         (OpCode == OpCode_BEQ || OpCode == OpCode_BNE || OpCode == OpCode_BLEZ ||
118          OpCode == OpCode_BGTZ || OpCode == OpCode_BLTZ) ? PCSrc_Branch :
119         (OpCode == OpCode_J || OpCode == OpCode_JAL) ? PCSrc_Jump :
120         (OpCode == OpCode_RType && (Funct == Funct_JR || Funct == Funct_JALR)) ?
    PCSrc_JumpR :
121         PCSrc_PCPlus4;
122     // Branch_Type: literally the type of branch instruction
123     assign Branch_Type =
124         (OpCode == OpCode_BEQ)  ? Branch_Type_BEQ :
125         (OpCode == OpCode_BNE)  ? Branch_Type_BNE :
126         (OpCode == OpCode_BLEZ) ? Branch_Type_BLEZ :
127         (OpCode == OpCode_BGTZ) ? Branch_Type_BGTZ :
128         (OpCode == OpCode_BLTZ) ? Branch_Type_BLTZ :
129         Branch_Type_BEQ;
130     // RegWrite: obviously whether to write to the register file
131     assign RegWrite =
132         (OpCode == OpCode_SW || OpCode == OpCode_BEQ || OpCode == OpCode_BNE ||
133          OpCode == OpCode_BLEZ || OpCode == OpCode_BGTZ || OpCode == OpCode_BLTZ ||
134          OpCode == OpCode_J || (OpCode == OpCode_RType && Funct == Funct_JR)) ? 1'b0 : //
    sw, beq, bne, blez, bgtz, bltz, j, jr
135         ((OpCode == OpCode_RType && Funct != Funct_JR) || OpCode == OpCode_LW ||
136          OpCode == OpCode_LUI || OpCode == OpCode_MUL || OpCode == OpCode_ADDI ||
137          OpCode == OpCode_ADDIU || OpCode == OpCode_ANDI || OpCode == OpCode_ORI ||
138          OpCode == OpCode_SLTI || OpCode == OpCode_SLTIU || OpCode == OpCode_JAL) ? 1'b1 :
```

```
    // R-type (-jr), lw, lui, mul, addi, addiu, andi, slti, sltiu, jal, jalr
139         1'b0; // others
140     // RegDst: R-type and jalr goes to Rd; I-type goes to Rt; jr needn't write back to
    register
141     assign RegDst =
142         ((OpCode == OpCode_RType && Funct != Funct_JR) || OpCode == OpCode_MUL) ?
    RegDst_RegRdAddr : // R-type (-jr), mul, jalr
143         (OpCode == OpCode_JAL) ? RegDst_RegRaAddr : // jal
144         (OpCode == OpCode_LW || OpCode == OpCode_LUI || OpCode == OpCode_ADDI ||
145          OpCode == OpCode_ADDIU || OpCode == OpCode_ANDI || OpCode == OpCode_ORI ||
146          OpCode == OpCode_SLTI || OpCode == OpCode_SLTIU) ? RegDst_RegRtAddr : // lw, lui,
    addi, addiu, andi, slti, sltiu
147         RegDst_RegNone; // others
148     // MemRead, MemWrite: only lw and sw need memory access
149     assign MemRead  = (OpCode == OpCode_LW) ? 1'b1 : 1'b0;
150     assign MemWrite = (OpCode == OpCode_SW) ? 1'b1 : 1'b0;
151     // MemtoReg: jump-type writes PC+4; lw writes memory data; R-type and some I-type write
     ALU output
152     assign MemtoReg =
153         (OpCode == OpCode_LW) ? MemtoReg_MemData : // lw
154         (OpCode == OpCode_JAL || (OpCode == OpCode_RType && Funct == Funct_JALR)) ?
    MemtoReg_PCPlus4 : // jal, jalr
155         ((OpCode == OpCode_RType && Funct != Funct_JR && Funct != Funct_JALR) || OpCode ==
    OpCode_MUL ||
156          OpCode == OpCode_LUI || OpCode == OpCode_ADDI || OpCode == OpCode_ADDIU ||
157          OpCode == OpCode_ANDI || OpCode == OpCode_ORI || OpCode == OpCode_SLTI ||
158          OpCode == OpCode_SLTIU) ? MemtoReg_ALUOut : // R-type (-jr/-jalr), mul, lui, addi,
     addiu, andi, slti, sltiu
159         MemtoReg_None; // others
160     // ALUSrcA: if shamt, then shamt; if Rs, then Rs; lui, jr, jalr needn't ALUSrcA
161     assign ALUSrcA =
162         (OpCode == OpCode_RType && (Funct == Funct_SLL || Funct == Funct_SRL ||
163          Funct == Funct_SRA)) ? ALUSrc_Shamt : // sll, srl, sra (all instructions with
    shamt)
164         ((OpCode == OpCode_RType && Funct != Funct_JR && Funct != Funct_JALR) || OpCode ==
    OpCode_LW ||
165          OpCode == OpCode_SW || OpCode == OpCode_MUL || OpCode == OpCode_ADDI ||
166          OpCode == OpCode_ADDIU || OpCode == OpCode_ANDI || OpCode == OpCode_ORI ||
167          OpCode == OpCode_SLTI || OpCode == OpCode_SLTIU || OpCode == OpCode_BEQ ||
168          OpCode == OpCode_BNE || OpCode == OpCode_BLEZ || OpCode == OpCode_BGTZ ||
169          OpCode == OpCode_BLTZ) ? ALUSrc_Reg : // R-type (-jr/-jalr), lw, sw, mul, addi,
```

```
addiu, andi, slti, sltiu, beq, blez, bgtz, bltz
        ALUSrc_None; // others
    // ALUSrcB: if imm, then immediate; if R-type and Rt isn't used as Rd, then Rt; branch-type's Rt==0 so both Rt and None is OK; jr, jalr needn't ALUSrcB
    assign ALUSrcB =
        (OpCode == OpCode_LW || OpCode == OpCode_SW || OpCode == OpCode_LUI ||
         OpCode == OpCode_ADDI || OpCode == OpCode_ADDIU || OpCode == OpCode_ANDI ||
         OpCode == OpCode_ORI || OpCode == OpCode_SLTI || OpCode == OpCode_SLTIU) ? ALUSrc_Imm : // lw, sw, lui, addi, addiu, andi, slti, sltiu
        ((OpCode == OpCode_RType && Funct != Funct_JR && Funct != Funct_JALR) ||
         OpCode == OpCode_MUL || OpCode == OpCode_BEQ || OpCode == OpCode_BNE ||
         OpCode == OpCode_BLEZ || OpCode == OpCode_BGTZ || OpCode == OpCode_BLTZ) ? ALUSrc_Reg : // R-type (-jr/-jalr), mul, beq, bne, blez, bgtz, bltz
        ALUSrc_None; // others
    // ExtOp: if lui, then LUIExtend; if andi, then ZeroExtend; if addi/addiu/slti/sltiu, then SignExtend; if 'offset', then SignExtend; shamt isn't extended here
    assign ExtOp =
        (OpCode == OpCode_LUI) ? ExtOp_LUIExtend : // lui
        (OpCode == OpCode_ANDI || OpCode == OpCode_ORI) ? ExtOp_ZeroExtend : // andi
        (OpCode == OpCode_ADDI || OpCode == OpCode_ADDIU || OpCode == OpCode_SLTI ||
          OpCode == OpCode_SLTIU || OpCode == OpCode_LW || OpCode == OpCode_SW ||
          OpCode == OpCode_BEQ || OpCode == OpCode_BNE || OpCode == OpCode_BLEZ ||
          OpCode == OpCode_BGTZ || OpCode == OpCode_BLTZ) ? ExtOp_SignExtend : // addi, addiu, slti, sltiu, lw, sw, beq, bne, blez, bgtz, bltz
        ExtOp_None; // others
    // ALUOp: jump-type needn't ALUOp; lui/lw/sw use ALUOp_ADD; branch-type uses ALUOp_SUB
    assign ALUOp =
        (OpCode == OpCode_RType) ? (
            (Funct == Funct_ADD || Funct == Funct_ADDU) ? ALUOp_ADD :
            (Funct == Funct_SUB || Funct == Funct_SUBU) ? ALUOp_SUB :
            (Funct == Funct_AND)                        ? ALUOp_AND :
            (Funct == Funct_OR)                         ? ALUOp_OR :
            (Funct == Funct_XOR)                        ? ALUOp_XOR :
            (Funct == Funct_NOR)                        ? ALUOp_NOR :
            (Funct == Funct_SLL)                        ? ALUOp_SLL :
            (Funct == Funct_SRL)                        ? ALUOp_SRL :
            (Funct == Funct_SRA)                        ? ALUOp_SRA :
            (Funct == Funct_SLT)                        ? ALUOp_SLT :
            (Funct == Funct_SLTU)                       ? ALUOp_SLTU :
            ALUOp_NOP
        ) :
```

```
205         (OpCode == OpCode_LW || OpCode == OpCode_SW || OpCode == OpCode_ADDI ||
206          OpCode == OpCode_ADDIU || OpCode == OpCode_LUI) ? ALUOp_ADD :
207         (OpCode == OpCode_MUL)   ? ALUOp_MUL  :
208         (OpCode == OpCode_ANDI)  ? ALUOp_AND  :
209         (OpCode == OpCode_ORI)   ? ALUOp_OR   :
210         (OpCode == OpCode_SLTI)  ? ALUOp_SLT  :
211         (OpCode == OpCode_SLTIU) ? ALUOp_SLTU :
212         (OpCode == OpCode_BEQ || OpCode == OpCode_BNE || OpCode == OpCode_BLEZ ||
213          OpCode == OpCode_BGTZ || OpCode == OpCode_BLTZ) ? ALUOp_SUB :
214         ALUOp_NOP;
215 endmodule
```

Listing 4: user/src/CPU.v

```
1  module CPU (
2      input wire clk,
3      input wire reset,
4
5      output wire [12 -1: 0] BCD7
6  );
7      // Do not use delay branching (jal: PC <= PC + 4)
8
9      // for simulation
10     `ifdef SIMULATION
11         parameter INST_FILE = "program.hex";
12     `endif
13
14     // PCSrc
15     parameter PCSrc_Branch  = 2'b11;
16     parameter PCSrc_Jump    = 2'b01;
17     parameter PCSrc_JumpR   = 2'b10;
18     parameter PCSrc_PCPlus4 = 2'b00;
19     // RegDst
20     parameter RegDst_RegRtAddr = 2'b11;
21     parameter RegDst_RegRdAddr = 2'b01;
22     parameter RegDst_RegRaAddr = 2'b10;
23     parameter RegDst_RegNone   = 2'b00;
24     // ALUSrc
25     parameter ALUSrc_Reg   = 2'b11; // ALUSrcA: Rs, ALUSrcB: Rt
26     parameter ALUSrc_Shamt = 2'b01;
27     parameter ALUSrc_Imm   = 2'b10;
```

```
28      parameter ALUSrc_None  = 2'b00;
29      // MemtoReg
30      parameter MemtoReg_MemData = 2'b11;
31      parameter MemtoReg_PCPlus4 = 2'b01;
32      parameter MemtoReg_ALUOut  = 2'b10;
33      parameter MemtoReg_None    = 2'b00;
34      // forwarding flags
35      parameter Forwarding_NONE  = 2'b00;
36      parameter Forwarding_EXMEM = 2'b01;
37      parameter Forwarding_MEMWB = 2'b10;
38
39      // Name: Bus_<OutputModule>_<OutputSignal>
40      // Hazard
41      wire            Bus_Hazard_PC_Write, Bus_Hazard_IFIDReg_Write, Bus_Hazard_IFIDReg_Flush
        , Bus_Hazard_IDEXReg_Write, Bus_Hazard_IDEXReg_Flush, Bus_Hazard_EXMEMReg_Write,
        Bus_Hazard_EXMEMReg_Flush, Bus_Hazard_MEMWBReg_Write, Bus_Hazard_MEMWBReg_Flush;
42      wire [ 2 -1: 0] Bus_Hazard_PCSrc;
43      // Forwarding
44      wire [ 2 -1: 0] Bus_Forwarding_Forward_ALUA, Bus_Forwarding_Forward_ALUB,
        Bus_Forwarding_Forward_IDRs, Bus_Forwarding_Forward_EXMEM_MEMWD,
        Bus_Forwarding_Forward_IDEX_MEMWD;
45      // PC
46      wire [32 -1: 0] Bus_PC_PC, Bus_PC_PCPlus4;
47      // IFIDRegs
48      wire [ 5 -1: 0] Bus_IFIDRegs_RegRsAddr, Bus_IFIDRegs_RegRtAddr, Bus_IFIDRegs_RegRdAddr,
         Bus_IFIDRegs_Shamt;
49      wire [ 6 -1: 0] Bus_IFIDRegs_OpCode, Bus_IFIDRegs_Funct;
50      wire [16 -1: 0] Bus_IFIDRegs_Imm16;
51      wire [26 -1: 0] Bus_IFIDRegs_Target26;
52      wire [32 -1: 0] Bus_IFIDRegs_PCPlus4;
53      // InstMem
54      wire [32 -1: 0] Bus_InstMem_Instruction;
55      // ControlUnit
56      wire            Bus_Control_RegWrite, Bus_Control_MemRead, Bus_Control_MemWrite;
57      wire [ 2 -1: 0] Bus_Control_PCSrc, Bus_Control_RegDst, Bus_Control_MemtoReg,
        Bus_Control_ALUSrcA, Bus_Control_ALUSrcB, Bus_Control_ExtOp;
58      wire [ 3 -1: 0] Bus_Control_Branch_Type;
59      wire [ 5 -1: 0] Bus_Control_ALUOp;
60      // ImmExtendUnit
61      wire [32 -1: 0] Bus_Immext_Imm;
62      // RegisterFile
```

```
63      wire [32 -1: 0] Bus_RegisterFile_ReadDataA, Bus_RegisterFile_ReadDataB;
64      // IDEXRegs
65      wire            Bus_IDEXRegs_RegWrite, Bus_IDEXRegs_MemRead, Bus_IDEXRegs_MemWrite;
66      wire [ 2 -1: 0] Bus_IDEXRegs_PCSrc, Bus_IDEXRegs_RegDst, Bus_IDEXRegs_MemtoReg,
    Bus_IDEXRegs_ALUSrcA, Bus_IDEXRegs_ALUSrcB;
67      wire [ 3 -1: 0] Bus_IDEXRegs_Branch_Type;
68      wire [ 5 -1: 0] Bus_IDEXRegs_ALUOp;
69      wire [ 5 -1: 0] Bus_IDEXRegs_RegRsAddr, Bus_IDEXRegs_RegRtAddr, Bus_IDEXRegs_RegRdAddr,
     Bus_IDEXRegs_Shamt;
70      wire [32 -1: 0] Bus_IDEXRegs_PCPlus4, Bus_IDEXRegs_RegRsData, Bus_IDEXRegs_RegRtData,
    Bus_IDEXRegs_ImmExtend;
71      // ALUControlUnit
72      wire [32 -1: 0] Bus_ALUControl_OutA, Bus_ALUControl_OutB;
73      // ALU
74      wire            Bus_ALU_Zero;
75      wire [32 -1: 0] Bus_ALU_ALUOut;
76      // EXMEMRegs
77      wire            Bus_EXMEMRegs_RegWrite, Bus_EXMEMRegs_MemRead, Bus_EXMEMRegs_MemWrite;
78      wire [ 2 -1: 0] Bus_EXMEMRegs_MemtoReg;
79      wire [ 5 -1: 0] Bus_EXMEMRegs_RegWrAddr, Bus_EXMEMRegs_RegRtAddr;
80      wire [32 -1: 0] Bus_EXMEMRegs_PCPlus4, Bus_EXMEMRegs_RegRtData, Bus_EXMEMRegs_ALUOut;
81      // DataMem
82      wire [12 -1: 0] Bus_DataMem_BCD7;
83      wire [32 -1: 0] Bus_DataMem_ReadData;
84      // MEMWBRegs
85      wire            Bus_MEMWBRegs_RegWrite;
86      wire [ 5 -1: 0] Bus_MEMWBRegs_RegWrAddr;
87      wire [32 -1: 0] Bus_MEMWBRegs_RegWriteData;
88
89      (* keep_hierarchy = "yes" *) PC u_PC (
90          .clk            (clk),
91          .reset          (reset),
92          .Jump_Target    ({Bus_IFIDRegs_PCPlus4[31: 28], Bus_IFIDRegs_Target26, 2'b00}),
93          .Branch_Target  (Bus_IDEXRegs_PCPlus4 + {Bus_IDEXRegs_ImmExtend[29: 0], 2'b00}),
94          .Jump_RegTarget (
95              (Bus_Forwarding_Forward_IDRs == Forwarding_EXMEM) ?
96                  ((Bus_EXMEMRegs_MemtoReg == MemtoReg_PCPlus4) ? Bus_EXMEMRegs_PCPlus4 :
97                   (Bus_EXMEMRegs_MemtoReg == MemtoReg_ALUOut)  ? Bus_EXMEMRegs_ALUOut  :
98                   32'h0000_0000) :
99              Bus_RegisterFile_ReadDataA
100         ),
```

```
101         .PCSrc          (Bus_Hazard_PCSrc),
102         .PC_Write       (Bus_Hazard_PC_Write),
103         .PC             (Bus_PC_PC),
104         .PC_Plus_4      (Bus_PC_PCPlus4)
105     );
106     // for simulation
107     `ifdef SIMULATION
108         InstMem #(
109             .INST_FILE(INST_FILE)
110         ) u_InstMem (
111             .Address     (Bus_PC_PC),
112             .Instruction (Bus_InstMem_Instruction)
113         );
114     `else
115     // for synthesis
116         (* keep_hierarchy = "yes" *) InstMem u_InstMem (
117             .Address     (Bus_PC_PC),
118             .Instruction (Bus_InstMem_Instruction)
119         );
120     `endif
121 
122     (* keep_hierarchy = "yes" *) IFIDRegs u_IFIDRegs (
123         .clk            (clk),
124         .reset          (reset),
125         .IFID_Write     (Bus_Hazard_IFIDReg_Write),
126         .IFID_Flush     (Bus_Hazard_IFIDReg_Flush),
127         .Instruction_In (Bus_InstMem_Instruction),
128         .PC_Plus_4_In   (Bus_PC_PCPlus4),
129         .PC_Plus_4_Out  (Bus_IFIDRegs_PCPlus4),
130         .OpCode_Out     (Bus_IFIDRegs_OpCode),
131         .RegRsAddr_Out  (Bus_IFIDRegs_RegRsAddr),
132         .RegRtAddr_Out  (Bus_IFIDRegs_RegRtAddr),
133         .RegRdAddr_Out  (Bus_IFIDRegs_RegRdAddr),
134         .Shamt_Out      (Bus_IFIDRegs_Shamt),
135         .Funct_Out      (Bus_IFIDRegs_Funct),
136         .Imm16_Out      (Bus_IFIDRegs_Imm16),
137         .Target26_Out   (Bus_IFIDRegs_Target26)
138     );
139     (* keep_hierarchy = "yes" *) ControlUnit u_ControlUnit (
140         .OpCode      (Bus_IFIDRegs_OpCode),
141         .Funct       (Bus_IFIDRegs_Funct),
```

```
142         .PCSrc       (Bus_Control_PCSrc),
143         .Branch_Type (Bus_Control_Branch_Type),
144         .RegWrite    (Bus_Control_RegWrite),
145         .RegDst      (Bus_Control_RegDst),
146         .MemRead     (Bus_Control_MemRead),
147         .MemWrite    (Bus_Control_MemWrite),
148         .MemtoReg    (Bus_Control_MemtoReg),
149         .ALUSrcA     (Bus_Control_ALUSrcA),
150         .ALUSrcB     (Bus_Control_ALUSrcB),
151         .ExtOp       (Bus_Control_ExtOp),
152         .ALUOp       (Bus_Control_ALUOp)
153     );
154
155     (* keep_hierarchy = "yes" *) RegisterFile u_RegisterFile (
156         .clk           (clk),
157         .reset         (reset),
158         .RegWrite      (Bus_MEMWBRegs_RegWrite),
159         .RegRead_AddrA (Bus_IFIDRegs_RegRsAddr),
160         .RegRead_AddrB (Bus_IFIDRegs_RegRtAddr),
161         .RegWrite_Addr (Bus_MEMWBRegs_RegWrAddr),
162         .RegWrite_Data (Bus_MEMWBRegs_RegWriteData),
163         .RegRead_DataA (Bus_RegisterFile_ReadDataA),
164         .RegRead_DataB (Bus_RegisterFile_ReadDataB)
165     );
166     (* keep_hierarchy = "yes" *) ImmExtendUnit u_ImmExtendUnit (
167         .Imm_In  (Bus_IFIDRegs_Imm16),
168         .ExtOp   (Bus_Control_ExtOp),
169         .Imm_Out (Bus_Immext_Imm)
170     );
171
172     (* keep_hierarchy = "yes" *) IDEXRegs u_IDEXRegs (
173         .clk             (clk),
174         .reset           (reset),
175         .IDEX_Write      (Bus_Hazard_IDEXReg_Write),
176         .IDEX_Flush      (Bus_Hazard_IDEXReg_Flush),
177         .PC_Plus_4_In    (Bus_IFIDRegs_PCPlus4),
178         .RegRsAddr_In    (Bus_IFIDRegs_RegRsAddr),
179         .RegRtAddr_In    (Bus_IFIDRegs_RegRtAddr),
180         .RegRdAddr_In    (Bus_IFIDRegs_RegRdAddr),
181         .RegRsData_In    (Bus_RegisterFile_ReadDataA),
182         .RegRtData_In    (Bus_RegisterFile_ReadDataB),
```

```
183         .Shamt_In        (Bus_IFIDRegs_Shamt),
184         .ImmExt_In       (Bus_Immext_Imm),
185         .PCSrc_In        (Bus_Control_PCSrc),
186         .Branch_Type_In  (Bus_Control_Branch_Type),
187         .RegWrite_In     (Bus_Control_RegWrite),
188         .RegDst_In       (Bus_Control_RegDst),
189         .MemRead_In      (Bus_Control_MemRead),
190         .MemWrite_In     (Bus_Control_MemWrite),
191         .MemtoReg_In     (Bus_Control_MemtoReg),
192         .ALUSrcA_In      (Bus_Control_ALUSrcA),
193         .ALUSrcB_In      (Bus_Control_ALUSrcB),
194         .ALUOp_In        (Bus_Control_ALUOp),
195         .PC_Plus_4_Out   (Bus_IDEXRegs_PCPlus4),
196         .RegRsAddr_Out   (Bus_IDEXRegs_RegRsAddr),
197         .RegRtAddr_Out   (Bus_IDEXRegs_RegRtAddr),
198         .RegRdAddr_Out   (Bus_IDEXRegs_RegRdAddr),
199         .RegRsData_Out   (Bus_IDEXRegs_RegRsData),
200         .RegRtData_Out   (Bus_IDEXRegs_RegRtData),
201         .Shamt_Out       (Bus_IDEXRegs_Shamt),
202         .ImmExt_Out      (Bus_IDEXRegs_ImmExtend),
203         .PCSrc_Out       (Bus_IDEXRegs_PCSrc),
204         .Branch_Type_Out (Bus_IDEXRegs_Branch_Type),
205         .RegWrite_Out    (Bus_IDEXRegs_RegWrite),
206         .RegDst_Out      (Bus_IDEXRegs_RegDst),
207         .MemRead_Out     (Bus_IDEXRegs_MemRead),
208         .MemWrite_Out    (Bus_IDEXRegs_MemWrite),
209         .MemtoReg_Out    (Bus_IDEXRegs_MemtoReg),
210         .ALUSrcA_Out     (Bus_IDEXRegs_ALUSrcA),
211         .ALUSrcB_Out     (Bus_IDEXRegs_ALUSrcB),
212         .ALUOp_Out       (Bus_IDEXRegs_ALUOp)
213     );
214     (* keep_hierarchy = "yes" *) ALUControlUnit u_ALUControlUnit (
215         .ALUSrcA              (Bus_IDEXRegs_ALUSrcA),
216         .ALUSrcB              (Bus_IDEXRegs_ALUSrcB),
217         .IDEX_RegRsData       (Bus_IDEXRegs_RegRsData),
218         .IDEX_RegRtData       (Bus_IDEXRegs_RegRtData),
219         .IDEX_Shamt           (Bus_IDEXRegs_Shamt),
220         .IDEX_ImmExtend       (Bus_IDEXRegs_ImmExtend),
221         .Forward_EXMEMRegData (
222             (Bus_EXMEMRegs_MemtoReg == MemtoReg_PCPlus4) ? Bus_EXMEMRegs_PCPlus4 :
223             (Bus_EXMEMRegs_MemtoReg == MemtoReg_ALUOut)  ? Bus_EXMEMRegs_ALUOut  :
```

```
            32'h0000_0000
        ),
        .Forward_MEMWBRegData (Bus_MEMWBRegs_RegWriteData),
        .ForwardingA          (Bus_Forwarding_Forward_ALUA),
        .ForwardingB          (Bus_Forwarding_Forward_ALUB),
        .OutA                 (Bus_ALUControl_OutA),
        .OutB                 (Bus_ALUControl_OutB)
    );
    (* keep_hierarchy = "yes" *) ALU u_ALU (
        .InA     (Bus_ALUControl_OutA),
        .InB     (Bus_ALUControl_OutB),
        .ALUOp   (Bus_IDEXRegs_ALUOp),
        .ALUOut  (Bus_ALU_ALUOut),
        .ALUZero (Bus_ALU_Zero)
    );

    (* keep_hierarchy = "yes" *) EXMEMRegs u_EXMEMRegs (
        .clk               (clk),
        .reset             (reset),
        .EXMEM_Write       (Bus_Hazard_EXMEMReg_Write),
        .EXMEM_Flush       (Bus_Hazard_EXMEMReg_Flush),
        .PC_Plus_4_In      (Bus_IDEXRegs_PCPlus4),
        .RegWrite_Addr_In  (
            (Bus_IDEXRegs_RegDst == RegDst_RegRtAddr) ? Bus_IDEXRegs_RegRtAddr :
            (Bus_IDEXRegs_RegDst == RegDst_RegRdAddr) ? Bus_IDEXRegs_RegRdAddr :
            (Bus_IDEXRegs_RegDst == RegDst_RegRaAddr) ? 5'b1_1111 : // $ra
            (Bus_IDEXRegs_RegDst == RegDst_RegNone)   ? 5'b0_0000 :
             5'b00000
        ),
        .ALUOut_In         (Bus_ALU_ALUOut),
        .RegWrite_In       (Bus_IDEXRegs_RegWrite),
        .MemRead_In        (Bus_IDEXRegs_MemRead),
        .MemWrite_In       (Bus_IDEXRegs_MemWrite),
        .RegRtAddr_In      (Bus_IDEXRegs_RegRtAddr),
        .RegRtData_In      (
            (Bus_Forwarding_Forward_IDEX_MEMWD == Forwarding_MEMWB) ?
            Bus_MEMWBRegs_RegWriteData : Bus_IDEXRegs_RegRtData
        ),
        .MemtoReg_In       (Bus_IDEXRegs_MemtoReg),
        .PC_Plus_4_Out     (Bus_EXMEMRegs_PCPlus4),
        .RegWrite_Out      (Bus_EXMEMRegs_RegWrite),
```

```
265         .RegWrite_Addr_Out (Bus_EXMEMRegs_RegWrAddr),
266         .MemRead_Out       (Bus_EXMEMRegs_MemRead),
267         .MemWrite_Out      (Bus_EXMEMRegs_MemWrite),
268         .RegRtAddr_Out     (Bus_EXMEMRegs_RegRtAddr),
269         .RegRtData_Out     (Bus_EXMEMRegs_RegRtData),
270         .MemtoReg_Out      (Bus_EXMEMRegs_MemtoReg),
271         .ALUOut_Out        (Bus_EXMEMRegs_ALUOut)
272     );
273     (* keep_hierarchy = "yes" *) DataMem u_DataMem(
274         .clk        (clk),
275         .reset      (reset),
276         .Address    (Bus_EXMEMRegs_ALUOut),
277         .WriteData  (
278             (Bus_Forwarding_Forward_EXMEM_MEMWD == Forwarding_MEMWB) ?
279             Bus_MEMWBRegs_RegWriteData : Bus_EXMEMRegs_RegRtData
280         ),
281         .MemWrite   (Bus_EXMEMRegs_MemWrite),
282         .MemRead    (Bus_EXMEMRegs_MemRead),
283         .ReadData   (Bus_DataMem_ReadData),
284         .BCD7       (BCD7)
285     );
286
287     (* keep_hierarchy = "yes" *) MEMWBRegs u_MEMWBRegs(
288         .clk               (clk),
289         .reset             (reset),
290         .MEMWB_Write       (Bus_Hazard_MEMWBReg_Write),
291         .MEMWB_Flush       (Bus_Hazard_MEMWBReg_Flush),
292         .PC_Plus_4_In      (Bus_EXMEMRegs_PCPlus4),
293         .RegWrite_In       (Bus_EXMEMRegs_RegWrite),
294         .RegWrite_Addr_In  (Bus_EXMEMRegs_RegWrAddr),
295         .MemData_In        (Bus_DataMem_ReadData),
296         .MemtoReg_In       (Bus_EXMEMRegs_MemtoReg),
297         .ALUOut_In         (Bus_EXMEMRegs_ALUOut),
298         .RegWrite_Addr_Out (Bus_MEMWBRegs_RegWrAddr),
299         .RegWriteData_Out  (Bus_MEMWBRegs_RegWriteData),
300         .RegWrite_Out      (Bus_MEMWBRegs_RegWrite)
301     );
302     (* keep_hierarchy = "yes" *) ForwardingUnit u_ForwardingUnit (
303         .EXMEM_RegWrite      (Bus_EXMEMRegs_RegWrite),
304         .EXMEM_RegWrAddr     (Bus_EXMEMRegs_RegWrAddr),
305         .MEMWB_RegWrite      (Bus_MEMWBRegs_RegWrite),
```

```
306         .MEMWB_RegWrAddr     (Bus_MEMWBRegs_RegWrAddr),
307         .IFID_PCSrc          (Bus_Control_PCSrc),
308         .IFID_RegRsAddr      (Bus_IFIDRegs_RegRsAddr),
309         .IDEX_RegRsAddr      (Bus_IDEXRegs_RegRsAddr),
310         .IDEX_RegRtAddr      (Bus_IDEXRegs_RegRtAddr),
311         .IDEX_MemWrite       (Bus_IDEXRegs_MemWrite),
312         .EXMEM_RegRtAddr     (Bus_EXMEMRegs_RegRtAddr),
313         .EXMEM_MemWrite      (Bus_EXMEMRegs_MemWrite),
314         .Forward_ALUA        (Bus_Forwarding_Forward_ALUA),
315         .Forward_ALUB        (Bus_Forwarding_Forward_ALUB),
316         .Forward_IDRs        (Bus_Forwarding_Forward_IDRs),
317         .Forward_EXMEM_MEMWD (Bus_Forwarding_Forward_EXMEM_MEMWD),
318         .Forward_IDEX_MEMWD  (Bus_Forwarding_Forward_IDEX_MEMWD)
319     );
320     (* keep_hierarchy = "yes" *) HazardUnit u_HazardUnit (
321         .IDEX_MemRead     (Bus_IDEXRegs_MemRead),
322         .IDEX_RegRtAddr   (Bus_IDEXRegs_RegRtAddr),
323         .IDEX_RegRdAddr   (Bus_IDEXRegs_RegRdAddr),
324         .IFID_RegRsAddr   (Bus_IFIDRegs_RegRsAddr),
325         .IFID_RegRtAddr   (Bus_IFIDRegs_RegRtAddr),
326         .EXMEM_MemtoReg   (Bus_EXMEMRegs_MemtoReg),
327         .IFID_PCSrc       (Bus_Control_PCSrc),
328         .IDEX_PCSrc       (Bus_IDEXRegs_PCSrc),
329         .IDEX_Branch_Type (Bus_IDEXRegs_Branch_Type),
330         .ALU_Out          (Bus_ALU_ALUOut),
331         .ALU_Zero         (Bus_ALU_Zero),
332         .PC_Write         (Bus_Hazard_PC_Write),
333         .PCSrc            (Bus_Hazard_PCSrc),
334         .IFID_Write       (Bus_Hazard_IFIDReg_Write),
335         .IDEX_Write       (Bus_Hazard_IDEXReg_Write),
336         .EXMEM_Write      (Bus_Hazard_EXMEMReg_Write),
337         .MEMWB_Write      (Bus_Hazard_MEMWBReg_Write),
338         .IFID_Flush       (Bus_Hazard_IFIDReg_Flush),
339         .IDEX_Flush       (Bus_Hazard_IDEXReg_Flush),
340         .EXMEM_Flush      (Bus_Hazard_EXMEMReg_Flush),
341         .MEMWB_Flush      (Bus_Hazard_MEMWBReg_Flush)
342     );
343 endmodule
```

Listing 5: user/src/Forwarding.v

```
module ForwardingUnit (
    input wire              EXMEM_RegWrite,
    input wire  [ 5 -1: 0] EXMEM_RegWrAddr,
    input wire              MEMWB_RegWrite,
    input wire  [ 5 -1: 0] MEMWB_RegWrAddr,
    input wire  [ 2 -1: 0] IFID_PCSrc,
    input wire  [ 5 -1: 0] IFID_RegRsAddr,
    input wire  [ 5 -1: 0] IDEX_RegRsAddr,
    input wire  [ 5 -1: 0] IDEX_RegRtAddr,
    input wire  [ 5 -1: 0] IDEX_MemWrite,
    input wire  [ 5 -1: 0] EXMEM_RegRtAddr,
    input wire              EXMEM_MemWrite,

    output wire [ 2 -1: 0] Forward_ALUA,
    output wire [ 2 -1: 0] Forward_ALUB,
    output wire [ 2 -1: 0] Forward_IDRs,
    output wire [ 2 -1: 0] Forward_EXMEM_MEMWD, // Forwarding to MEM Write Data in
EXMEMRegs
    output wire [ 2 -1: 0] Forward_IDEX_MEMWD // Forwarding to MEM Write Data in IDEXRegs
);
    // forwarding flags
    parameter Forwarding_NONE  = 2'b00;
    parameter Forwarding_EXMEM = 2'b01;
    parameter Forwarding_MEMWB = 2'b10;
    // PCSrc
    parameter PCSrc_Branch  = 2'b11;
    parameter PCSrc_Jump    = 2'b01;
    parameter PCSrc_JumpR   = 2'b10;
    parameter PCSrc_PCPlus4 = 2'b00;

    assign Forward_ALUA =
        (EXMEM_RegWrite && EXMEM_RegWrAddr == IDEX_RegRsAddr && EXMEM_RegWrAddr != 5'h00) ?
 Forwarding_EXMEM :
        (MEMWB_RegWrite && MEMWB_RegWrAddr == IDEX_RegRsAddr && MEMWB_RegWrAddr != 5'h00) ?
 Forwarding_MEMWB :
        Forwarding_NONE;
    assign Forward_ALUB =
        (EXMEM_RegWrite && EXMEM_RegWrAddr == IDEX_RegRtAddr && EXMEM_RegWrAddr != 5'h00) ?
 Forwarding_EXMEM :
        (MEMWB_RegWrite && MEMWB_RegWrAddr == IDEX_RegRtAddr && MEMWB_RegWrAddr != 5'h00) ?
 Forwarding_MEMWB :
```

```
37          Forwarding_NONE;
38      assign Forward_EXMEM_MEMWD =
39          (MEMWB_RegWrite && MEMWB_RegWrAddr == EXMEM_RegRtAddr && MEMWB_RegWrAddr != 5'h00
    && EXMEM_MemWrite) ? Forwarding_MEMWB :
40          Forwarding_NONE;
41      assign Forward_IDEX_MEMWD =
42          (MEMWB_RegWrite && MEMWB_RegWrAddr == IDEX_RegRtAddr && MEMWB_RegWrAddr != 5'h00 &&
     IDEX_MemWrite) ? Forwarding_MEMWB :
43          Forwarding_NONE;
44      assign Forward_IDRs =
45          (EXMEM_RegWrite && EXMEM_RegWrAddr == IFID_RegRsAddr && EXMEM_RegWrAddr != 5'h00 &&
     IFID_PCSrc == PCSrc_JumpR) ? Forwarding_EXMEM :
46          Forwarding_NONE;
47  endmodule
```

Listing 6: user/src/Hazard.v

```
1   module HazardUnit (
2       // Load-use hazard
3       input wire              IDEX_MemRead,
4       input wire  [ 5 -1: 0] IDEX_RegRtAddr,
5       input wire  [ 5 -1: 0] IDEX_RegRdAddr,
6       input wire  [ 5 -1: 0] IFID_RegRsAddr,
7       input wire  [ 5 -1: 0] IFID_RegRtAddr,
8       input wire  [ 2 -1: 0] EXMEM_MemtoReg,
9       // Control hazard
10      input wire  [ 2 -1: 0] IFID_PCSrc,
11      input wire  [ 2 -1: 0] IDEX_PCSrc,
12      input wire  [ 3 -1: 0] IDEX_Branch_Type,
13      input wire  [32 -1: 0] ALU_Out,
14      input wire              ALU_Zero,
15
16      output wire [ 2 -1: 0] PCSrc,
17      output wire             PC_Write,
18      output wire             IFID_Write,
19      output wire             IDEX_Write,
20      output wire             EXMEM_Write,
21      output wire             MEMWB_Write,
22      output wire             IFID_Flush,
23      output wire             IDEX_Flush,
24      output wire             EXMEM_Flush,
```

```
25      output wire             MEMWB_Flush
26  );
27      // Load-use hazard should be prioritized over control  hazard
28      // Load-use hazard is not necessarily detected in the ID stage, but can be detected in
    the EX stage
29      // PCSrc
30      parameter PCSrc_Branch  = 2'b11;
31      parameter PCSrc_Jump    = 2'b01;
32      parameter PCSrc_JumpR   = 2'b10;
33      parameter PCSrc_PCPlus4 = 2'b00;
34      // Branch Type
35      parameter Branch_Type_NONE = 3'b000;
36      parameter Branch_Type_BEQ  = 3'b101;
37      parameter Branch_Type_BNE  = 3'b001;
38      parameter Branch_Type_BLEZ = 3'b010;
39      parameter Branch_Type_BGTZ = 3'b011;
40      parameter Branch_Type_BLTZ = 3'b100;
41      // MemtoReg
42      parameter MemtoReg_MemData = 2'b11;
43      parameter MemtoReg_PCPlus4 = 2'b01;
44      parameter MemtoReg_ALUOut  = 2'b10;
45      parameter MemtoReg_None    = 2'b00;
46
47      wire Should_Branch = (IDEX_PCSrc == PCSrc_Branch) &&
48                           ((IDEX_Branch_Type == Branch_Type_BEQ && ALU_Zero) ||
49                            (IDEX_Branch_Type == Branch_Type_BNE && ~ALU_Zero) ||
50                            (IDEX_Branch_Type == Branch_Type_BLEZ && (ALU_Out[31] || ALU_Zero
    )) ||
51                            (IDEX_Branch_Type == Branch_Type_BGTZ && (~ALU_Out[31] && ~
    ALU_Zero)) ||
52                            (IDEX_Branch_Type == Branch_Type_BLTZ && ALU_Out[31]));
53      wire EX_Branch = (IDEX_PCSrc == PCSrc_Branch && Should_Branch);
54      wire ID_Jump   = (IFID_PCSrc == PCSrc_Jump);
55      wire ID_JumpR  = (IFID_PCSrc == PCSrc_JumpR);
56
57      assign {PCSrc, PC_Write} =
58          // Load-use hazard
59          (IDEX_MemRead && (IDEX_RegRtAddr == IFID_RegRsAddr || IDEX_RegRtAddr ==
    IFID_RegRtAddr)) ? {PCSrc_PCPlus4, 1'b0} :
60          (IFID_PCSrc == PCSrc_JumpR && (IFID_RegRsAddr == IDEX_RegRtAddr || IFID_RegRsAddr
    == IDEX_RegRdAddr)) ? {PCSrc_PCPlus4, 1'b0} :
```

```
61          (IFID_PCSrc == PCSrc_JumpR && (EXMEM_MemtoReg == MemtoReg_MemData)) ? {
    PCSrc_PCPlus4, 1'b0} :
62          // Control hazard
63          (EX_Branch) ? {PCSrc_Branch, 1'b1} :
64          (ID_Jump || ID_JumpR) ? {IFID_PCSrc, 1'b1} :
65          // No hazard
66          {PCSrc_PCPlus4, 1'b1} ;
67      assign {IFID_Write, IDEX_Write, EXMEM_Write, MEMWB_Write} =
68          // Load-use hazard
69          (IDEX_MemRead && (IDEX_RegRtAddr == IFID_RegRsAddr || IDEX_RegRtAddr ==
    IFID_RegRtAddr)) ? 4'b0011 :
70          (IFID_PCSrc == PCSrc_JumpR && (IFID_RegRsAddr == IDEX_RegRtAddr || IFID_RegRsAddr
    == IDEX_RegRdAddr)) ? 4'b0011 :
71          (IFID_PCSrc == PCSrc_JumpR && (EXMEM_MemtoReg == MemtoReg_MemData)) ? 4'b0011 :
72          // Control hazard
73          (EX_Branch)           ? 4'b0011 :
74          (ID_Jump || ID_JumpR) ? 4'b0111 :
75          // No hazard
76          4'b1111 ;
77      assign {IFID_Flush, IDEX_Flush, EXMEM_Flush, MEMWB_Flush} =
78          // Load-use hazard
79          (IDEX_MemRead && (IDEX_RegRtAddr == IFID_RegRsAddr || IDEX_RegRtAddr ==
    IFID_RegRtAddr)) ? 4'b0100 :
80          (IFID_PCSrc == PCSrc_JumpR && (IFID_RegRsAddr == IDEX_RegRtAddr || IFID_RegRsAddr
    == IDEX_RegRdAddr)) ? 4'b0100 :
81          (IFID_PCSrc == PCSrc_JumpR && (EXMEM_MemtoReg == MemtoReg_MemData)) ? 4'b0100 :
82          // Control hazard
83          (EX_Branch)           ? 4'b1100 :
84          (ID_Jump || ID_JumpR) ? 4'b1000 :
85          // No hazard
86          4'b0000 ;
87  endmodule
```

Listing 7: user/src/Memory.v

```
1   module InstMem (
2       input wire  [32 -1: 0] Address,
3       output reg  [32 -1: 0] Instruction
4   );
5       // for simulation
6       `ifdef SIMULATION
```

```
7         parameter INST_FILE = "program.hex";
8         reg [32 -1: 0] rom [0: 255];
9         initial begin
10            $readmemh(INST_FILE, rom);
11        end
12        assign Instruction = (Address <= 32'h0040_03FC) ?
13            rom[Address[9: 2]] : 32'h0000_0000;
14    // for synthesis
15    `else
16        always @(*) begin
17            case (Address)
18                32'h0040_0000: Instruction = 32'h2403_0000;
19                32'h0040_0004: Instruction = 32'h0c10_006d;
20                32'h0040_0008: Instruction = 32'h2403_0001;
21                32'h0040_000c: Instruction = 32'h0000_4021;
22                32'h0040_0010: Instruction = 32'h0102_082a;
23                32'h0040_0014: Instruction = 32'h1020_00ad;
24                32'h0040_0018: Instruction = 32'h2001_0004;
25                32'h0040_001c: Instruction = 32'h7101_5802;
26                32'h0040_0020: Instruction = 32'h216b_0100;
27                32'h0040_0024: Instruction = 32'h8d6b_0000;
28                32'h0040_0028: Instruction = 32'h2409_0000;
29                32'h0040_002c: Instruction = 32'h2921_0b15;
30                32'h0040_0030: Instruction = 32'h1020_005e;
31                32'h0040_0034: Instruction = 32'h240a_0000;
32                32'h0040_0038: Instruction = 32'h0009_6780;
33                32'h0040_003c: Instruction = 32'h3c01_c000;
34                32'h0040_0040: Instruction = 32'h3421_0000;
35                32'h0040_0044: Instruction = 32'h102c_000f;
36                32'h0040_0048: Instruction = 32'h3c01_8000;
37                32'h0040_004c: Instruction = 32'h3421_0000;
38                32'h0040_0050: Instruction = 32'h102c_0009;
39                32'h0040_0054: Instruction = 32'h3c01_4000;
40                32'h0040_0058: Instruction = 32'h3421_0000;
41                32'h0040_005c: Instruction = 32'h102c_0003;
42                32'h0040_0060: Instruction = 32'h240c_0100;
43                32'h0040_0064: Instruction = 32'h000b_6b02;
44                32'h0040_0068: Instruction = 32'h0810_0023;
45                32'h0040_006c: Instruction = 32'h240c_0200;
46                32'h0040_0070: Instruction = 32'h000b_6a02;
47                32'h0040_0074: Instruction = 32'h0810_0023;
```

```
48                  32'h0040_0078: Instruction = 32'h240c_0400;
49                  32'h0040_007c: Instruction = 32'h000b_6902;
50                  32'h0040_0080: Instruction = 32'h0810_0023;
51                  32'h0040_0084: Instruction = 32'h240c_0800;
52                  32'h0040_0088: Instruction = 32'h000b_6802;
53                  32'h0040_008c: Instruction = 32'h31ad_000f;
54                  32'h0040_0090: Instruction = 32'h2001_0000;
55                  32'h0040_0094: Instruction = 32'h102d_003a;
56                  32'h0040_0098: Instruction = 32'h2001_0001;
57                  32'h0040_009c: Instruction = 32'h102d_0036;
58                  32'h0040_00a0: Instruction = 32'h2001_0002;
59                  32'h0040_00a4: Instruction = 32'h102d_0032;
60                  32'h0040_00a8: Instruction = 32'h2001_0003;
61                  32'h0040_00ac: Instruction = 32'h102d_002e;
62                  32'h0040_00b0: Instruction = 32'h2001_0004;
63                  32'h0040_00b4: Instruction = 32'h102d_002a;
64                  32'h0040_00b8: Instruction = 32'h2001_0005;
65                  32'h0040_00bc: Instruction = 32'h102d_0026;
66                  32'h0040_00c0: Instruction = 32'h2001_0006;
67                  32'h0040_00c4: Instruction = 32'h102d_0022;
68                  32'h0040_00c8: Instruction = 32'h2001_0007;
69                  32'h0040_00cc: Instruction = 32'h102d_001e;
70                  32'h0040_00d0: Instruction = 32'h2001_0008;
71                  32'h0040_00d4: Instruction = 32'h102d_001a;
72                  32'h0040_00d8: Instruction = 32'h2001_0009;
73                  32'h0040_00dc: Instruction = 32'h102d_0016;
74                  32'h0040_00e0: Instruction = 32'h2001_000a;
75                  32'h0040_00e4: Instruction = 32'h102d_0012;
76                  32'h0040_00e8: Instruction = 32'h2001_000b;
77                  32'h0040_00ec: Instruction = 32'h102d_000e;
78                  32'h0040_00f0: Instruction = 32'h2001_000c;
79                  32'h0040_00f4: Instruction = 32'h102d_000a;
80                  32'h0040_00f8: Instruction = 32'h2001_000d;
81                  32'h0040_00fc: Instruction = 32'h102d_0006;
82                  32'h0040_0100: Instruction = 32'h2001_000e;
83                  32'h0040_0104: Instruction = 32'h102d_0002;
84                  32'h0040_0108: Instruction = 32'h240d_0071;
85                  32'h0040_010c: Instruction = 32'h0810_0061;
86                  32'h0040_0110: Instruction = 32'h240d_0079;
87                  32'h0040_0114: Instruction = 32'h0810_0061;
88                  32'h0040_0118: Instruction = 32'h240d_00bf;
```

```
89                  32'h0040_011c: Instruction = 32'h0810_0061;
90                  32'h0040_0120: Instruction = 32'h240d_0039;
91                  32'h0040_0124: Instruction = 32'h0810_0061;
92                  32'h0040_0128: Instruction = 32'h240d_00ff;
93                  32'h0040_012c: Instruction = 32'h0810_0061;
94                  32'h0040_0130: Instruction = 32'h240d_0077;
95                  32'h0040_0134: Instruction = 32'h0810_0061;
96                  32'h0040_0138: Instruction = 32'h240d_006f;
97                  32'h0040_013c: Instruction = 32'h0810_0061;
98                  32'h0040_0140: Instruction = 32'h240d_007f;
99                  32'h0040_0144: Instruction = 32'h0810_0061;
100                 32'h0040_0148: Instruction = 32'h240d_0007;
101                 32'h0040_014c: Instruction = 32'h0810_0061;
102                 32'h0040_0150: Instruction = 32'h240d_007d;
103                 32'h0040_0154: Instruction = 32'h0810_0061;
104                 32'h0040_0158: Instruction = 32'h240d_006d;
105                 32'h0040_015c: Instruction = 32'h0810_0061;
106                 32'h0040_0160: Instruction = 32'h240d_0066;
107                 32'h0040_0164: Instruction = 32'h0810_0061;
108                 32'h0040_0168: Instruction = 32'h240d_004f;
109                 32'h0040_016c: Instruction = 32'h0810_0061;
110                 32'h0040_0170: Instruction = 32'h240d_005b;
111                 32'h0040_0174: Instruction = 32'h0810_0061;
112                 32'h0040_0178: Instruction = 32'h240d_0006;
113                 32'h0040_017c: Instruction = 32'h0810_0061;
114                 32'h0040_0180: Instruction = 32'h240d_003f;
115                 32'h0040_0184: Instruction = 32'h018d_6025;
116                 32'h0040_0188: Instruction = 32'h3c01_4000;
117                 32'h0040_018c: Instruction = 32'h0020_0821;
118                 32'h0040_0190: Instruction = 32'hac2c_0010;
119                 32'h0040_0194: Instruction = 32'h2941_0c00;
120                 32'h0040_0198: Instruction = 32'h1020_0002;
121                 32'h0040_019c: Instruction = 32'h214a_0001;
122                 32'h0040_01a0: Instruction = 32'h0810_0065;
123                 32'h0040_01a4: Instruction = 32'h2129_0001;
124                 32'h0040_01a8: Instruction = 32'h0810_000b;
125                 32'h0040_01ac: Instruction = 32'h2108_0001;
126                 32'h0040_01b0: Instruction = 32'h0810_0004;
127                 32'h0040_01b4: Instruction = 32'h2408_0000;
128                 32'h0040_01b8: Instruction = 32'h8d10_0000;
129                 32'h0040_01bc: Instruction = 32'h8d11_0004;
```

```
130                 32'h0040_01c0: Instruction = 32'h8d12_0008;
131                 32'h0040_01c4: Instruction = 32'h8d13_000c;
132                 32'h0040_01c8: Instruction = 32'h2114_0010;
133                 32'h0040_01cc: Instruction = 32'h0013_a880;
134                 32'h0040_01d0: Instruction = 32'h02b4_a820;
135                 32'h0040_01d4: Instruction = 32'h0013_b0c0;
136                 32'h0040_01d8: Instruction = 32'h02d4_b020;
137                 32'h0040_01dc: Instruction = 32'h2217_0001;
138                 32'h0040_01e0: Instruction = 32'h0017_b880;
139                 32'h0040_01e4: Instruction = 32'h02f6_b820;
140                 32'h0040_01e8: Instruction = 32'h2408_0000;
141                 32'h0040_01ec: Instruction = 32'h7212_1802;
142                 32'h0040_01f0: Instruction = 32'h0103_082a;
143                 32'h0040_01f4: Instruction = 32'h1020_0007;
144                 32'h0040_01f8: Instruction = 32'h2409_0100;
145                 32'h0040_01fc: Instruction = 32'h2001_0004;
146                 32'h0040_0200: Instruction = 32'h7101_5002;
147                 32'h0040_0204: Instruction = 32'h012a_4820;
148                 32'h0040_0208: Instruction = 32'had20_0000;
149                 32'h0040_020c: Instruction = 32'h2108_0001;
150                 32'h0040_0210: Instruction = 32'h0810_007c;
151                 32'h0040_0214: Instruction = 32'h240c_0000;
152                 32'h0040_0218: Instruction = 32'h0190_082a;
153                 32'h0040_021c: Instruction = 32'h1020_0029;
154                 32'h0040_0220: Instruction = 32'h2001_0004;
155                 32'h0040_0224: Instruction = 32'h7181_c002;
156                 32'h0040_0228: Instruction = 32'h02d8_c020;
157                 32'h0040_022c: Instruction = 32'h8f0b_0000;
158                 32'h0040_0230: Instruction = 32'h8f18_0004;
159                 32'h0040_0234: Instruction = 32'h000b_6820;
160                 32'h0040_0238: Instruction = 32'h01b8_082a;
161                 32'h0040_023c: Instruction = 32'h1020_001f;
162                 32'h0040_0240: Instruction = 32'h2001_0004;
163                 32'h0040_0244: Instruction = 32'h71a1_7002;
164                 32'h0040_0248: Instruction = 32'h02ae_7020;
165                 32'h0040_024c: Instruction = 32'h8dce_0000;
166                 32'h0040_0250: Instruction = 32'h2001_0004;
167                 32'h0040_0254: Instruction = 32'h71a1_c802;
168                 32'h0040_0258: Instruction = 32'h0299_c820;
169                 32'h0040_025c: Instruction = 32'h8f39_0000;
170                 32'h0040_0260: Instruction = 32'h240f_0000;
```

```
171                 32'h0040_0264: Instruction = 32'h01f2_082a;
172                 32'h0040_0268: Instruction = 32'h1020_0012;
173                 32'h0040_026c: Instruction = 32'h7192_4802;
174                 32'h0040_0270: Instruction = 32'h012f_4820;
175                 32'h0040_0274: Instruction = 32'h2001_0004;
176                 32'h0040_0278: Instruction = 32'h7121_4802;
177                 32'h0040_027c: Instruction = 32'h240b_0100;
178                 32'h0040_0280: Instruction = 32'h012b_4820;
179                 32'h0040_0284: Instruction = 32'h71d2_5002;
180                 32'h0040_0288: Instruction = 32'h014f_5020;
181                 32'h0040_028c: Instruction = 32'h2001_0004;
182                 32'h0040_0290: Instruction = 32'h7141_5002;
183                 32'h0040_0294: Instruction = 32'h0157_5020;
184                 32'h0040_0298: Instruction = 32'h8d2b_0000;
185                 32'h0040_029c: Instruction = 32'h8d4a_0000;
186                 32'h0040_02a0: Instruction = 32'h7159_5002;
187                 32'h0040_02a4: Instruction = 32'h016a_5820;
188                 32'h0040_02a8: Instruction = 32'had2b_0000;
189                 32'h0040_02ac: Instruction = 32'h21ef_0001;
190                 32'h0040_02b0: Instruction = 32'h0810_0099;
191                 32'h0040_02b4: Instruction = 32'h21ad_0001;
192                 32'h0040_02b8: Instruction = 32'h0810_008e;
193                 32'h0040_02bc: Instruction = 32'h218c_0001;
194                 32'h0040_02c0: Instruction = 32'h0810_0086;
195                 32'h0040_02c4: Instruction = 32'h7212_1002;
196                 32'h0040_02c8: Instruction = 32'h03e0_0008;
197                 32'h0040_02cc: Instruction = 32'h2403_0000;
198                 32'h0040_02d0: Instruction = 32'h2418_ffff;
199                 32'h0040_02d4: Instruction = 32'h0018_c180;
200                 32'h0040_02d8: Instruction = 32'h0318_c027;
201                 32'h0040_02dc: Instruction = 32'h0003_c982;
202                 32'h0040_02e0: Instruction = 32'h0338_c824;
203                 32'h0040_02e4: Instruction = 32'h0019_c902;
204                 32'h0040_02e8: Instruction = 32'h2001_0001;
205                 32'h0040_02ec: Instruction = 32'h1039_0009;
206                 32'h0040_02f0: Instruction = 32'h2001_0002;
207                 32'h0040_02f4: Instruction = 32'h1039_000c;
208                 32'h0040_02f8: Instruction = 32'h2001_0003;
209                 32'h0040_02fc: Instruction = 32'h1039_000f;
210                 32'h0040_0300: Instruction = 32'h240c_0279;
211                 32'h0040_0304: Instruction = 32'h3c01_4000;
```

```
212                 32'h0040_0308: Instruction = 32'h0020_0821;
213                 32'h0040_030c: Instruction = 32'hac2c_0010;
214                 32'h0040_0310: Instruction = 32'h0810_00d4;
215                 32'h0040_0314: Instruction = 32'h240c_0454;
216                 32'h0040_0318: Instruction = 32'h3c01_4000;
217                 32'h0040_031c: Instruction = 32'h0020_0821;
218                 32'h0040_0320: Instruction = 32'hac2c_0010;
219                 32'h0040_0324: Instruction = 32'h0810_00d4;
220                 32'h0040_0328: Instruction = 32'h240c_085e;
221                 32'h0040_032c: Instruction = 32'h3c01_4000;
222                 32'h0040_0330: Instruction = 32'h0020_0821;
223                 32'h0040_0334: Instruction = 32'hac2c_0010;
224                 32'h0040_0338: Instruction = 32'h0810_00d4;
225                 32'h0040_033c: Instruction = 32'h240c_0000;
226                 32'h0040_0340: Instruction = 32'h3c01_4000;
227                 32'h0040_0344: Instruction = 32'h0020_0821;
228                 32'h0040_0348: Instruction = 32'hac2c_0010;
229                 32'h0040_034c: Instruction = 32'h0810_00d4;
230                 32'h0040_0350: Instruction = 32'h2063_0001;
231                 32'h0040_0354: Instruction = 32'h0810_00b7;
232                 32'h0040_0358: Instruction = 32'h0000_0000;
233                 32'h0040_035c: Instruction = 32'h0000_0000;
234                 32'h0040_0360: Instruction = 32'h0000_0000;
235                 32'h0040_0364: Instruction = 32'h0000_0000;
236                 32'h0040_0368: Instruction = 32'h0000_0000;
237                 32'h0040_036c: Instruction = 32'h0000_0000;
238                 32'h0040_0370: Instruction = 32'h0000_0000;
239                 32'h0040_0374: Instruction = 32'h0000_0000;
240                 32'h0040_0378: Instruction = 32'h0000_0000;
241                 32'h0040_037c: Instruction = 32'h0000_0000;
242                 32'h0040_0380: Instruction = 32'h0000_0000;
243                 32'h0040_0384: Instruction = 32'h0000_0000;
244                 32'h0040_0388: Instruction = 32'h0000_0000;
245                 32'h0040_038c: Instruction = 32'h0000_0000;
246                 32'h0040_0390: Instruction = 32'h0000_0000;
247                 32'h0040_0394: Instruction = 32'h0000_0000;
248                 32'h0040_0398: Instruction = 32'h0000_0000;
249                 32'h0040_039c: Instruction = 32'h0000_0000;
250                 32'h0040_03a0: Instruction = 32'h0000_0000;
251                 32'h0040_03a4: Instruction = 32'h0000_0000;
252                 32'h0040_03a8: Instruction = 32'h0000_0000;
```

```
253                 32'h0040_03ac: Instruction = 32'h0000_0000;
254                 32'h0040_03b0: Instruction = 32'h0000_0000;
255                 32'h0040_03b4: Instruction = 32'h0000_0000;
256                 32'h0040_03b8: Instruction = 32'h0000_0000;
257                 32'h0040_03bc: Instruction = 32'h0000_0000;
258                 32'h0040_03c0: Instruction = 32'h0000_0000;
259                 32'h0040_03c4: Instruction = 32'h0000_0000;
260                 32'h0040_03c8: Instruction = 32'h0000_0000;
261                 32'h0040_03cc: Instruction = 32'h0000_0000;
262                 32'h0040_03d0: Instruction = 32'h0000_0000;
263                 32'h0040_03d4: Instruction = 32'h0000_0000;
264                 32'h0040_03d8: Instruction = 32'h0000_0000;
265                 32'h0040_03dc: Instruction = 32'h0000_0000;
266                 32'h0040_03e0: Instruction = 32'h0000_0000;
267                 32'h0040_03e4: Instruction = 32'h0000_0000;
268                 32'h0040_03e8: Instruction = 32'h0000_0000;
269                 32'h0040_03ec: Instruction = 32'h0000_0000;
270                 32'h0040_03f0: Instruction = 32'h0000_0000;
271                 32'h0040_03f4: Instruction = 32'h0000_0000;
272                 32'h0040_03f8: Instruction = 32'h0000_0000;
273                 32'h0040_03fc: Instruction = 32'h0000_0000;
274                 default:       Instruction = 32'h0000_0000;
275             endcase
276         end
277     `endif
278 endmodule
279 
280 module DataMem (
281     input wire              clk,
282     input wire              reset,
283     input wire  [32 -1: 0] Address,
284     input wire  [32 -1: 0] WriteData,
285     input wire              MemWrite,
286     input wire              MemRead,
287 
288     output wire [32 -1: 0] ReadData,
289     output wire [12 -1: 0] BCD7
290 );
291     reg [32 -1: 0] memory [0: 128 -1]; // Data memory with 512 words
292     reg [12 -1: 0] BCD7_Reg; // BCD7 register for 0x4000_0010
293     assign BCD7 = BCD7_Reg;
```

```
294     assign ReadData =
295         {MemRead, Address[31: 24]} == 9'h100 ? memory[Address[10: 2]] : 32'h0000_0000;
296     integer i;
297 
298     always @(posedge clk) begin
299         if (reset) begin
300             BCD7_Reg <= 12'h000;
301             `ifdef NOT_SPARSE_MATMUL
302             // Fxxk vivado, it cannot find `define in parent .v file
303             // If I `define SPARSE_MATMUL in tb_CPU_inst_3.v (testbench for matmul process), iverilog can successfully go into `ifdef SPARSE_MATMUL here, but vivado cannot
304             // So here's the way around. Simulating using iverilog and define NOT_SPARSE_MATMUL in tb_CPU_inst_1.v (testbench for basic instructions, not matmul), then iverilog can go into `ifdef NOT_SPARSE_MATMUL here; When synthesizing using vivado, vivado will not define NOT_SPARSE_MATMUL, so it can go into `else` branch
305                 for (i = 0; i < 128; i = i + 1) begin
306                     memory[i] <= 32'h0000_0000;
307                 end
308             `else
309                 for (i = 36; i < 128; i = i + 1) begin
310                     memory[i] <= 32'h0000_0000;
311                 end
312                 memory[ 0] <= 32'h0000_0003;
313                 memory[ 1] <= 32'h0000_0004;
314                 memory[ 2] <= 32'h0000_0005;
315                 memory[ 3] <= 32'h0000_0004;
316                 memory[ 4] <= 32'h0000_0009;
317                 memory[ 5] <= 32'h0000_0007;
318                 memory[ 6] <= 32'h0000_000f;
319                 memory[ 7] <= 32'h0000_0009;
320                 memory[ 8] <= 32'h0000_0002;
321                 memory[ 9] <= 32'h0000_0001;
322                 memory[10] <= 32'h0000_0000;
323                 memory[11] <= 32'h0000_0002;
324                 memory[12] <= 32'h0000_0000;
325                 memory[13] <= 32'h0000_0001;
326                 memory[14] <= 32'h0000_0002;
327                 memory[15] <= 32'h0000_0004;
328                 memory[16] <= 32'h0000_0001;
329                 memory[17] <= 32'h0000_0004;
330                 memory[18] <= 32'h0000_0000;
```

```
                memory[19] <= 32'h0000_000c;
                memory[20] <= 32'h0000_000b;
                memory[21] <= 32'h0000_0009;
                memory[22] <= 32'h0000_0000;
                memory[23] <= 32'h0000_000b;
                memory[24] <= 32'h0000_0008;
                memory[25] <= 32'h0000_0002;
                memory[26] <= 32'h0000_000c;
                memory[27] <= 32'h0000_0002;
                memory[28] <= 32'h0000_000b;
                memory[29] <= 32'h0000_000a;
                memory[30] <= 32'h0000_0000;
                memory[31] <= 32'h0000_000a;
                memory[32] <= 32'h0000_000c;
                memory[33] <= 32'h0000_0000;
                memory[34] <= 32'h0000_0001;
                memory[35] <= 32'h0000_0009;
            `endif
        end
        else if (MemWrite) begin
            case (Address[31: 24])
                // Data memory
                8'h00: memory[Address[10: 2]] <= WriteData;
                // Peripheral device
                8'h40: begin
                    case (Address)
                        32'h4000_0010: BCD7_Reg <= WriteData[11: 0];
                        default: BCD7_Reg <= BCD7_Reg;
                    endcase
                end
            endcase
        end
    end
endmodule
```

Listing 8: user/src/PC.v

```
module PC (
    input wire             clk,
    input wire             reset,
    input wire  [26 -1: 0] Jump_Target,
```

```
    input wire  [32 -1: 0] Branch_Target,
    input wire  [32 -1: 0] Jump_RegTarget,
    input wire  [ 2 -1: 0] PCSrc,
    input wire             PC_Write,

    output wire [32 -1: 0] PC,
    output wire [32 -1: 0] PC_Plus_4
);
    // Jump_Target and Branch_Target have already benn calculated
    // PCSrc_Branch (EX) is prioritized over PCSrc_Jump (ID) and PCSrc_JumpR (ID)
    // PC INIT
    parameter PC_INIT = 32'h0040_0000;
    // PCSrc
    parameter PCSrc_Branch  = 2'b11;
    parameter PCSrc_Jump    = 2'b01;
    parameter PCSrc_JumpR   = 2'b10;
    parameter PCSrc_PCPlus4 = 2'b00;

    reg [32 -1: 0] PC_reg;
    assign PC = PC_reg;
    assign PC_Plus_4 = PC_reg + 32'd4;

    always @(posedge clk) begin
        if (reset) begin
            PC_reg <= PC_INIT;
        end
        else if (PC_Write) begin
            case (PCSrc)
                PCSrc_Branch:  PC_reg <= Branch_Target;
                PCSrc_Jump:    PC_reg <= Jump_Target;
                PCSrc_JumpR:   PC_reg <= Jump_RegTarget;
                PCSrc_PCPlus4: PC_reg <= PC_Plus_4;
                default:       PC_reg <= PC_Plus_4;
            endcase
        end
        else begin
            PC_reg <= PC_reg;
        end
    end
endmodule
```

Listing 9: user/src/Register.v

```
module RegisterFile (
    input wire              clk,
    input wire              reset,
    input wire              RegWrite,
    input wire  [ 5 -1: 0] RegRead_AddrA,
    input wire  [ 5 -1: 0] RegRead_AddrB,
    input wire  [ 5 -1: 0] RegWrite_Addr,
    input wire  [32 -1: 0] RegWrite_Data,

    output wire [32 -1: 0] RegRead_DataA,
    output wire [32 -1: 0] RegRead_DataB
);
    reg [32 -1: 0] registers [32 -1: 0];
    assign RegRead_DataA = (RegRead_AddrA == 5'b0_0000) ? 32'h0000_0000 : registers[RegRead_AddrA];
    assign RegRead_DataB = (RegRead_AddrB == 5'b0_0000) ? 32'h0000_0000 : registers[RegRead_AddrB];
    integer i;

    always @(negedge clk) begin
        if (reset) begin
            for (i = 0; i < 32; i = i + 1) begin
                registers[i] <= 32'h0000_0000;
            end
        end
        if (RegWrite && RegWrite_Addr != 5'b0_0000) begin
            registers[RegWrite_Addr] <= RegWrite_Data;
        end
    end
endmodule

module IFIDRegs (
    input wire              clk,
    input wire              reset,
    input wire              IFID_Write,
    input wire              IFID_Flush,
    input wire  [32 -1: 0] Instruction_In,
    input wire  [32 -1: 0] PC_Plus_4_In,

    output wire [32 -1: 0] PC_Plus_4_Out,
```

```
39      output wire [ 6 -1: 0] OpCode_Out,
40      output wire [ 5 -1: 0] RegRsAddr_Out,
41      output wire [ 5 -1: 0] RegRtAddr_Out,
42      output wire [ 5 -1: 0] RegRdAddr_Out,
43      output wire [ 5 -1: 0] Shamt_Out,
44      output wire [ 6 -1: 0] Funct_Out,
45      output wire [16 -1: 0] Imm16_Out,
46      output wire [26 -1: 0] Target26_Out
47  );
48      reg [32 -1: 0] PC_Plus_4_Reg;
49      reg [ 6 -1: 0] OpCode_Reg;
50      reg [ 5 -1: 0] RegRsAddr_Reg;
51      reg [ 5 -1: 0] RegRtAddr_Reg;
52      reg [ 5 -1: 0] RegRdAddr_Reg;
53      reg [ 5 -1: 0] Shamt_Reg;
54      reg [ 6 -1: 0] Funct_Reg;
55      reg [16 -1: 0] Imm16_Reg;
56      reg [26 -1: 0] Target26_Reg;
57      assign PC_Plus_4_Out = PC_Plus_4_Reg;
58      assign OpCode_Out    = OpCode_Reg;
59      assign RegRsAddr_Out = RegRsAddr_Reg;
60      assign RegRtAddr_Out = RegRtAddr_Reg;
61      assign RegRdAddr_Out = RegRdAddr_Reg;
62      assign Shamt_Out     = Shamt_Reg;
63      assign Funct_Out     = Funct_Reg;
64      assign Imm16_Out     = Imm16_Reg;
65      assign Target26_Out  = Target26_Reg;
66
67      // IFID_Flush is prioritized over IFID_Write
68      always @(posedge clk) begin
69          if (reset || IFID_Flush) begin
70              PC_Plus_4_Reg <= 32'h0000_0000;
71              OpCode_Reg    <= 6'h00;
72              RegRsAddr_Reg <= 5'h00;
73              RegRtAddr_Reg <= 5'h00;
74              RegRdAddr_Reg <= 5'h00;
75              Shamt_Reg     <= 5'h00;
76              Funct_Reg     <= 6'h00;
77              Imm16_Reg     <= 16'h0000;
78              Target26_Reg  <= 26'h0000000;
79          end
```

```
80          else if (IFID_Write) begin
81              PC_Plus_4_Reg <= PC_Plus_4_In;
82              OpCode_Reg    <= Instruction_In[31: 26];
83              RegRsAddr_Reg <= Instruction_In[25: 21];
84              RegRtAddr_Reg <= Instruction_In[20: 16];
85              RegRdAddr_Reg <= Instruction_In[15: 11];
86              Shamt_Reg     <= Instruction_In[10:  6];
87              Funct_Reg     <= Instruction_In[ 5:  0];
88              Imm16_Reg     <= Instruction_In[15:  0];
89              Target26_Reg  <= Instruction_In[25:  0];
90          end
91      end
92  endmodule
93
94  module IDEXRegs (
95      input wire              clk,
96      input wire              reset,
97      input wire              IDEX_Write,
98      input wire              IDEX_Flush,
99
100     input wire  [32 -1: 0] PC_Plus_4_In,
101     input wire  [ 5 -1: 0] RegRsAddr_In,
102     input wire  [ 5 -1: 0] RegRtAddr_In,
103     input wire  [ 5 -1: 0] RegRdAddr_In,
104     input wire  [32 -1: 0] RegRsData_In,
105     input wire  [32 -1: 0] RegRtData_In,
106     input wire  [ 5 -1: 0] Shamt_In,
107     input wire  [32 -1: 0] ImmExt_In,
108
109     input wire  [ 2 -1: 0] PCSrc_In,
110     input wire  [ 3 -1: 0] Branch_Type_In,
111     input wire             RegWrite_In,
112     input wire  [ 2 -1: 0] RegDst_In,
113     input wire             MemRead_In,
114     input wire             MemWrite_In,
115     input wire  [ 2 -1: 0] MemtoReg_In,
116     input wire  [ 2 -1: 0] ALUSrcA_In,
117     input wire  [ 2 -1: 0] ALUSrcB_In,
118     input wire  [ 5 -1: 0] ALUOp_In,
119
120     output wire [32 -1: 0] PC_Plus_4_Out,
```

```
121     output wire [ 5 -1: 0] RegRsAddr_Out,
122     output wire [ 5 -1: 0] RegRtAddr_Out,
123     output wire [ 5 -1: 0] RegRdAddr_Out,
124     output wire [32 -1: 0] RegRsData_Out,
125     output wire [32 -1: 0] RegRtData_Out,
126     output wire [ 5 -1: 0] Shamt_Out,
127     output wire [32 -1: 0] ImmExt_Out,
128
129     output wire [ 2 -1: 0] PCSrc_Out,
130     output wire [ 3 -1: 0] Branch_Type_Out,
131     output wire            RegWrite_Out,
132     output wire [ 2 -1: 0] RegDst_Out,
133     output wire            MemRead_Out,
134     output wire            MemWrite_Out,
135     output wire [ 2 -1: 0] MemtoReg_Out,
136     output wire [ 2 -1: 0] ALUSrcA_Out,
137     output wire [ 2 -1: 0] ALUSrcB_Out,
138     output wire [ 5 -1: 0] ALUOp_Out
139 );
140     reg [32 -1: 0] PC_Plus_4_Reg;
141     reg [ 5 -1: 0] RegRsAddr_Reg;
142     reg [ 5 -1: 0] RegRtAddr_Reg;
143     reg [ 5 -1: 0] RegRdAddr_Reg;
144     reg [32 -1: 0] RegRsData_Reg;
145     reg [32 -1: 0] RegRtData_Reg;
146     reg [ 5 -1: 0] Shamt_Reg;
147     reg [32 -1: 0] ImmExt_Reg;
148     reg [ 2 -1: 0] PCSrc_Reg;
149     reg [ 3 -1: 0] Branch_Type_Reg;
150     reg            RegWrite_Reg;
151     reg [ 2 -1: 0] RegDst_Reg;
152     reg            MemRead_Reg;
153     reg            MemWrite_Reg;
154     reg [ 2 -1: 0] MemtoReg_Reg;
155     reg [ 2 -1: 0] ALUSrcA_Reg;
156     reg [ 2 -1: 0] ALUSrcB_Reg;
157     reg [ 5 -1: 0] ALUOp_Reg;
158     assign PC_Plus_4_Out   = PC_Plus_4_Reg;
159     assign RegRsAddr_Out   = RegRsAddr_Reg;
160     assign RegRtAddr_Out   = RegRtAddr_Reg;
161     assign RegRdAddr_Out   = RegRdAddr_Reg;
```

```
162     assign RegRsData_Out   = RegRsData_Reg;
163     assign RegRtData_Out   = RegRtData_Reg;
164     assign Shamt_Out       = Shamt_Reg;
165     assign ImmExt_Out      = ImmExt_Reg;
166     assign PCSrc_Out       = PCSrc_Reg;
167     assign Branch_Type_Out = Branch_Type_Reg;
168     assign RegWrite_Out    = RegWrite_Reg;
169     assign RegDst_Out      = RegDst_Reg;
170     assign MemRead_Out     = MemRead_Reg;
171     assign MemWrite_Out    = MemWrite_Reg;
172     assign MemtoReg_Out    = MemtoReg_Reg;
173     assign ALUSrcA_Out     = ALUSrcA_Reg;
174     assign ALUSrcB_Out     = ALUSrcB_Reg;
175     assign ALUOp_Out       = ALUOp_Reg;
176
177     // IDEX_Flush is prioritized over IDEX_Write
178     always @(posedge clk) begin
179         if (reset || IDEX_Flush) begin
180             PC_Plus_4_Reg   <= 32'h0000_0000;
181             RegRsAddr_Reg   <= 5'h00;
182             RegRtAddr_Reg   <= 5'h00;
183             RegRdAddr_Reg   <= 5'h00;
184             RegRsData_Reg   <= 32'h0000_0000;
185             RegRtData_Reg   <= 32'h0000_0000;
186             Shamt_Reg       <= 5'h00;
187             ImmExt_Reg      <= 32'h0000_0000;
188             PCSrc_Reg       <= 2'b00;
189             Branch_Type_Reg <= 3'b000;
190             RegWrite_Reg    <= 1'b0;
191             RegDst_Reg      <= 2'b00;
192             MemRead_Reg     <= 1'b0;
193             MemWrite_Reg    <= 1'b0;
194             MemtoReg_Reg    <= 2'b00;
195             ALUSrcA_Reg     <= 2'b00;
196             ALUSrcB_Reg     <= 2'b00;
197             ALUOp_Reg       <= 4'b0000;
198         end
199         else if (IDEX_Write) begin
200             PC_Plus_4_Reg   <= PC_Plus_4_In;
201             RegRsAddr_Reg   <= RegRsAddr_In;
202             RegRtAddr_Reg   <= RegRtAddr_In;
```

```
203             RegRdAddr_Reg    <= RegRdAddr_In;
204             RegRsData_Reg    <= RegRsData_In;
205             RegRtData_Reg    <= RegRtData_In;
206             Shamt_Reg        <= Shamt_In;
207             ImmExt_Reg       <= ImmExt_In;
208             PCSrc_Reg        <= PCSrc_In;
209             Branch_Type_Reg  <= Branch_Type_In;
210             RegWrite_Reg     <= RegWrite_In;
211             RegDst_Reg       <= RegDst_In;
212             MemRead_Reg      <= MemRead_In;
213             MemWrite_Reg     <= MemWrite_In;
214             MemtoReg_Reg     <= MemtoReg_In;
215             ALUSrcA_Reg      <= ALUSrcA_In;
216             ALUSrcB_Reg      <= ALUSrcB_In;
217             ALUOp_Reg        <= ALUOp_In;
218         end
219     end
220 endmodule
221
222 module EXMEMRegs (
223     input wire              clk,
224     input wire              reset,
225     input wire              EXMEM_Write,
226     input wire              EXMEM_Flush,
227
228     input wire  [32 -1: 0] PC_Plus_4_In,
229     input wire              RegWrite_In,
230     input wire  [ 5 -1: 0] RegWrite_Addr_In,
231     input wire  [ 5 -1: 0] RegRtAddr_In,
232     input wire  [32 -1: 0] RegRtData_In,
233     input wire              MemRead_In,
234     input wire              MemWrite_In,
235     input wire  [ 2 -1: 0] MemtoReg_In,
236     input wire  [32 -1: 0] ALUOut_In,
237
238     output wire [32 -1: 0] PC_Plus_4_Out,
239     output wire             RegWrite_Out,
240     output wire [ 5 -1: 0] RegWrite_Addr_Out,
241     output wire [ 5 -1: 0] RegRtAddr_Out,
242     output wire [32 -1: 0] RegRtData_Out,
243     output wire             MemRead_Out,
```

```
244     output wire             MemWrite_Out,
245     output wire [ 2 -1: 0] MemtoReg_Out,
246     output wire [32 -1: 0] ALUOut_Out
247 );
248     reg [32 -1: 0] PC_Plus_4_Reg;
249     reg            RegWrite_Reg;
250     reg [ 5 -1: 0] RegWrite_Addr_Reg;
251     reg [ 5 -1: 0] RegRtAddr_Reg;
252     reg [32 -1: 0] RegRtData_Reg;
253     reg            MemRead_Reg;
254     reg            MemWrite_Reg;
255     reg [ 2 -1: 0] MemtoReg_Reg;
256     reg [32 -1: 0] ALUOut_Reg;
257     assign PC_Plus_4_Out     = PC_Plus_4_Reg;
258     assign RegWrite_Out      = RegWrite_Reg;
259     assign RegWrite_Addr_Out = RegWrite_Addr_Reg;
260     assign MemRead_Out       = MemRead_Reg;
261     assign MemWrite_Out      = MemWrite_Reg;
262     assign RegRtAddr_Out     = RegRtAddr_Reg;
263     assign RegRtData_Out     = RegRtData_Reg;
264     assign MemtoReg_Out      = MemtoReg_Reg;
265     assign ALUOut_Out        = ALUOut_Reg;
266
267     // EXMEM_Flush is prioritized over EXMEM_Write
268     always @(posedge clk) begin
269         if (reset || EXMEM_Flush) begin
270             PC_Plus_4_Reg     <= 32'h0000_0000;
271             RegWrite_Reg      <= 1'b0;
272             RegWrite_Addr_Reg <= 5'h00;
273             MemRead_Reg       <= 1'b0;
274             MemWrite_Reg      <= 1'b0;
275             RegRtAddr_Reg     <= 32'h0000_0000;
276             RegRtData_Reg     <= 32'h0000_0000;
277             MemtoReg_Reg      <= 2'b00;
278             ALUOut_Reg        <= 32'h0000_0000;
279         end
280         else if (EXMEM_Write) begin
281             PC_Plus_4_Reg     <= PC_Plus_4_In;
282             RegWrite_Reg      <= RegWrite_In;
283             RegWrite_Addr_Reg <= RegWrite_Addr_In;
284             MemRead_Reg       <= MemRead_In;
```

```
285             MemWrite_Reg      <= MemWrite_In;
286             RegRtAddr_Reg     <= RegRtAddr_In;
287             RegRtData_Reg     <= RegRtData_In;
288             MemtoReg_Reg      <= MemtoReg_In;
289             ALUOut_Reg        <= ALUOut_In;
290         end
291     end
292 endmodule
293 
294 module MEMWBRegs (
295     input wire             clk,
296     input wire             reset,
297     input wire             MEMWB_Write,
298     input wire             MEMWB_Flush,
299 
300     input wire  [32 -1: 0] PC_Plus_4_In,
301     input wire             RegWrite_In,
302     input wire  [ 5 -1: 0] RegWrite_Addr_In,
303     input wire  [32 -1: 0] MemData_In,
304     input wire  [ 2 -1: 0] MemtoReg_In,
305     input wire  [32 -1: 0] ALUOut_In,
306 
307     output wire            RegWrite_Out,
308     output wire [ 5 -1: 0] RegWrite_Addr_Out,
309     output wire [32 -1: 0] RegWriteData_Out
310 );
311     // MemtoReg
312     parameter MemtoReg_MemData = 2'b11;
313     parameter MemtoReg_PCPlus4 = 2'b01;
314     parameter MemtoReg_ALUOut  = 2'b10;
315     parameter MemtoReg_None    = 2'b00;
316 
317     reg             RegWrite_Reg;
318     reg  [ 5 -1: 0] RegWrite_Addr_Reg;
319     reg  [32 -1: 0] RegWriteData_Reg;
320     wire [32 -1: 0] RegWriteData_Wire =
321         (MemtoReg_In == MemtoReg_MemData) ? MemData_In :
322         (MemtoReg_In == MemtoReg_PCPlus4) ? PC_Plus_4_In :
323         (MemtoReg_In == MemtoReg_ALUOut)  ? ALUOut_In :
324         32'h0000_0000;
325     assign RegWrite_Out      = RegWrite_Reg;
```

```
326     assign RegWrite_Addr_Out = RegWrite_Addr_Reg;
327     assign RegWriteData_Out  = RegWriteData_Reg;
328 
329     // MEMWB_Flush is prioritized over MEMWB_Write
330     always @(posedge clk) begin
331         if (reset || MEMWB_Flush) begin
332             RegWrite_Reg      <= 1'b0;
333             RegWrite_Addr_Reg <= 5'h00;
334             RegWriteData_Reg  <= 32'h0000_0000;
335         end
336         else if (MEMWB_Write) begin
337             RegWrite_Reg      <= RegWrite_In;
338             RegWrite_Addr_Reg <= RegWrite_Addr_In;
339             RegWriteData_Reg  <= RegWriteData_Wire;
340         end
341     end
342 endmodule
```

Listing 10: user/src/sparse_matmul.asm

```
1     # do the function
2     li      $v1,            0                       # $v1 = 0: symbol of doing
    multiplication
3     jal     sparse_matmul                           # jump to the sparse matrix
    multiplication function
4 
5     # print the result by BCD LEDs
6     li      $v1,            1                       # $v1 = 1: symbol of doing
    printing
7     move    $t0,            $zero                   # $t0: outest loop variable
8 mat_loop:                                           # outest loop, controlling the
    sequence of the whole matrix
9     bge     $t0,            $v0,            end_program
10 
11    mul     $t3,            $t0,            4
12    addi    $t3,            $t3,            0x00000100
13    lw      $t3,            ($t3)                   # $t3: the very number to be
    printed this time
14    li      $t1,            0                       # $t1: middle loop variable
15 time_loop:                                         # middle loop, controlling the
    sequence of four positions' enabling
```

```
16      bge     $t1,            2837,           end_time    # taking turns of printing each
    position for 4096 loops
17      li      $t2,            0
18      sll     $t4,            $t1,            30          # an_i = 1 while other an's =
    0, if i = $t4 (mod 4), $t4: lit position
19      beq     $t4,            0xc0000000,     an3
20      beq     $t4,            0x80000000,     an2
21      beq     $t4,            0x40000000,     an1
22      # only last two bytes are useful, while first two bytes are always zero and ignored
23  an0:
24      li      $t4,            0x0100
25      srl     $t5,            $t3,            12
26      j       bcd
27  an1:
28      li      $t4,            0x0200
29      srl     $t5,            $t3,            8
30      j       bcd
31  an2:
32      li      $t4,            0x0400
33      srl     $t5,            $t3,            4
34      j       bcd
35  an3:
36      li      $t4,            0x0800
37      srl     $t5,            $t3,            0
38
39  bcd:                                                # $t5: the number to be shown
40      andi    $t5,            $t5,            0xf
41      beq     $t5,            0x0,            is0
42      beq     $t5,            0x1,            is1
43      beq     $t5,            0x2,            is2
44      beq     $t5,            0x3,            is3
45      beq     $t5,            0x4,            is4
46      beq     $t5,            0x5,            is5
47      beq     $t5,            0x6,            is6
48      beq     $t5,            0x7,            is7
49      beq     $t5,            0x8,            is8
50      beq     $t5,            0x9,            is9
51      beq     $t5,            0xa,            isa
52      beq     $t5,            0xb,            isb
53      beq     $t5,            0xc,            isc
54      beq     $t5,            0xd,            isd
```

```
    beq     $t5,            0xe,                ise
    # $t5: now to be the lit LED indices
isf:
    li      $t5,            0x71
    j       light
ise:
    li      $t5,            0x79
    j       light
isd:
    li      $t5,            0xbf
    j       light
isc:
    li      $t5,            0x39
    j       light
isb:
    li      $t5,            0xff
    j       light
isa:
    li      $t5,            0x77
    j       light
is9:
    li      $t5,            0x6f
    j       light
is8:
    li      $t5,            0x7f
    j       light
is7:
    li      $t5,            0x07
    j       light
is6:
    li      $t5,            0x7d
    j       light
is5:
    li      $t5,            0x6d
    j       light
is4:
    li      $t5,            0x66
    j       light
is3:
    li      $t5,            0x4f
    j       light
```

```
96  is2:
97      li      $t5,            0x5b
98      j       light
99  is1:
100     li      $t5,            0x06
101     j       light
102 is0:
103     li      $t5,            0x3f
104
105 light:                                          # $t4: to be the full code to
    be saved in memory, instructing LED lighting
106     or      $t4,            $t4,                $t5
107     sw      $t4,            0x40000010($zero)
108
109 show_loop:                                      # innest loop, waiting with the
     same status
110     bge     $t2,            4096,               end_show    # $t2: innest loop variable
111     addi    $t2,            $t2,                1           # $t2++;
112     j       show_loop
113
114 end_show:
115     addi    $t1,            $t1,                1           # $t1++;
116     j       time_loop
117
118 end_time:
119     addi    $t0,            $t0,                1           # $t0++;
120     j       mat_loop
121
122     # the main function of sparse matrix multiplication, search in the assembly homework
    for more detail
123 sparse_matmul:
124     la      $t0,            0x00000000
125     lw      $s0,            0x00000000($t0)             # $s0: m
126     lw      $s1,            0x00000004($t0)             # $s1: n
127     lw      $s2,            0x00000008($t0)             # $s2: p
128     lw      $s3,            0x0000000c($t0)             # $s3: s
129
130     addi    $s4,            $t0,                16          # $s4: values
131
132     sll     $s5,            $s3,                2
133     add     $s5,            $s5,                $s4         # $s5: col_indices
```

```
134
135     sll     $s6,            $s3,            3
136     add     $s6,            $s6,            $s4         # $s6: row_ptr
137
138     addi    $s7,            $s0,            1
139     sll     $s7,            $s7,            2
140     add     $s7,            $s7,            $s6         # $s7: B
141     # set zeros
142     li      $t0,            0                           # int i = 0;
143     mul     $v1,            $s0,            $s2         # $v1: m * p
144     loop_0  :
145     bge     $t0,            $v1,            end_loop_0  # if (i < m * p) next,
    otherwise to end_loop_0
146
147     la      $t1,            0x00000100                  # $t1: C
148     mul     $t2,            $t0,            4           # $t2: (int*)%i;
149     add     $t1,            $t1,            $t2         # $t1 = C + i;
150     sw      $zero,          0x00000000($t1)             # C[i] = 0;
151
152     addi    $t0,            $t0,            1           # i++;
153     j       loop_0                                      # go back to the beginning of
    the loop
154 end_loop_0:
155
156     # the big loop, with $t3 used multiple times in different loops
157     # t3: start, t4: i, t5: j, t6: k, t7: l, t8: end, t9: val
158     li      $t4,            0                           # int i = 0;
159     loop_3  :
160     bge     $t4,            $s0,            end_loop_3  # if (i < m) next, otherwise to
     end_loop_3
161
162     mul     $t8,            $t4,            4           # end = (int*)i;
163     add     $t8,            $s6,            $t8         # end = row_ptr[i];
164     lw      $t3,            0x00000000($t8)             # start = row_ptr[i];
165     lw      $t8,            0x00000004($t8)             # end = row_ptr[i + 1]
166
167     # middle loop
168     add     $t5,            $zero,          $t3         # int j = start;
169     loop_2  :
170     bge     $t5,            $t8,            end_loop_2  # if (j < end) next, otherwise
    to end_loop_2
```

```
171
172     mul     $t6,            $t5,            4           # k = (int*)j;
173     add     $t6,            $s5,            $t6         # k = &col_indices[j];
174     lw      $t6,            0x00000000($t6)             # k = col_indices[j];
175
176     mul     $t9,            $t5,            4           # val = (int*)j;
177     add     $t9,            $s4,            $t9         # val = &values[j];
178     lw      $t9,            0x00000000($t9)             # val = values[j];
179
180     #inner loop
181     li      $t7,            0                           # int l = 0;
182     loop_1  :
183     bge     $t7,            $s2,            end_loop_1  # if (l < p) next, otherwise to
    end_loop_1
184
185     mul     $t1,            $t4,            $s2         # $t1: i * p;
186     add     $t1,            $t1,            $t7         # $t1 = i * p + l;
187     mul     $t1,            $t1,            4           # (int*)$t1;
188     la      $t3,            0x00000100                  # $t3: C
189     add     $t1,            $t1,            $t3         # $t1 = &C[i * p + l];
190
191     mul     $t2,            $t6,            $s2         # $t2: k * p;
192     add     $t2,            $t2,            $t7         # $t2 = k * p + l;
193     mul     $t2,            $t2,            4           # (int*)$t2;
194     add     $t2,            $t2,            $s7         # $t2 = &B[k * p + l];
195
196     lw      $t3,            0x00000000($t1)             # $t3: c_val = C[i * p + l];
197     lw      $t2,            0x00000000($t2)             # $t2 = B[k * p + l];
198     mul     $t2,            $t2,            $t9         # $t2 = val * B[k * p + l];
199     add     $t3,            $t3,            $t2         # c_val = c_val + val * B[k * p
    + l];
200     sw      $t3,            0x00000000($t1)             # C[i * p + l] = c_val;
201
202     addi    $t7,            $t7,            1           # l++;
203     j       loop_1                                      # go back to the beginning of
    the loop
204 end_loop_1:
205     addi    $t5,            $t5,            1           # j++;
206     j       loop_2                                      # go back to the beginning of
    the loop
207 end_loop_2:
```

```
208
209     addi    $t4,            $t4,            1           # i++;
210     j       loop_3                                      # go back to the beginning of
    the loop
211 end_loop_3:
212     mul     $v0,            $s0,            $s2         # calculate m * p;
213     jr      $ra                                         # return m * p;
214
215     # some ending animation, printing "_End" forever
216 end_program:
217     li      $v1,            0                           # $v1 = 0: ending, $v1 is also
    the loop variable
218     li      $t8,            -1                          # $t8: getting valid positions
    in $t9
219     sll     $t8,            $t8,            6           # 2^6 * T is the loop cycle
220     nor     $t8,            $t8,            $t8
221 denoument:
222     srl     $t9,            $v1,            6           # $t9: $v1 mod 2^6
223     and     $t9,            $t9,            $t8
224     srl     $t9,            $t9,            4           # divide the loop cycle into
    four identical periods
225     beq     $t9,            1,              end1
226     beq     $t9,            2,              end2
227     beq     $t9,            3,              end3
228 end0:
229     li      $t4,            0x00000279
230     sw      $t4,            0x40000010($zero)           # _E__
231     j       end_denoument
232 end1:
233     li      $t4,            0x00000454
234     sw      $t4,            0x40000010($zero)           # __n_
235     j       end_denoument
236 end2:
237     li      $t4,            0x0000085e
238     sw      $t4,            0x40000010($zero)           # ___d
239     j       end_denoument
240 end3:
241     li      $t4,            0
242     sw      $t4,            0x40000010($zero)           # -___
243     j       end_denoument
244 end_denoument:
```

```
245     addi    $v1,            $v1,            1           # $v1++;
246     j       denoument
```

Listing 11: user/src/sparse_matmul.inst

```
1  32'h0040_0000: Instruction = 32'h2403_0000;
2  32'h0040_0004: Instruction = 32'h0c10_006d;
3  32'h0040_0008: Instruction = 32'h2403_0001;
4  32'h0040_000c: Instruction = 32'h0000_4021;
5  32'h0040_0010: Instruction = 32'h0102_082a;
6  32'h0040_0014: Instruction = 32'h1020_00ad;
7  32'h0040_0018: Instruction = 32'h2001_0004;
8  32'h0040_001c: Instruction = 32'h7101_5802;
9  32'h0040_0020: Instruction = 32'h216b_0100;
10 32'h0040_0024: Instruction = 32'h8d6b_0000;
11 32'h0040_0028: Instruction = 32'h2409_0000;
12 32'h0040_002c: Instruction = 32'h2921_0b15;
13 32'h0040_0030: Instruction = 32'h1020_005e;
14 32'h0040_0034: Instruction = 32'h240a_0000;
15 32'h0040_0038: Instruction = 32'h0009_6780;
16 32'h0040_003c: Instruction = 32'h3c01_c000;
17 32'h0040_0040: Instruction = 32'h3421_0000;
18 32'h0040_0044: Instruction = 32'h102c_000f;
19 32'h0040_0048: Instruction = 32'h3c01_8000;
20 32'h0040_004c: Instruction = 32'h3421_0000;
21 32'h0040_0050: Instruction = 32'h102c_0009;
22 32'h0040_0054: Instruction = 32'h3c01_4000;
23 32'h0040_0058: Instruction = 32'h3421_0000;
24 32'h0040_005c: Instruction = 32'h102c_0003;
25 32'h0040_0060: Instruction = 32'h240c_0100;
26 32'h0040_0064: Instruction = 32'h000b_6b02;
27 32'h0040_0068: Instruction = 32'h0810_0023;
28 32'h0040_006c: Instruction = 32'h240c_0200;
29 32'h0040_0070: Instruction = 32'h000b_6a02;
30 32'h0040_0074: Instruction = 32'h0810_0023;
31 32'h0040_0078: Instruction = 32'h240c_0400;
32 32'h0040_007c: Instruction = 32'h000b_6902;
33 32'h0040_0080: Instruction = 32'h0810_0023;
34 32'h0040_0084: Instruction = 32'h240c_0800;
35 32'h0040_0088: Instruction = 32'h000b_6802;
36 32'h0040_008c: Instruction = 32'h31ad_000f;
```

```
32'h0040_0090: Instruction = 32'h2001_0000;
32'h0040_0094: Instruction = 32'h102d_003a;
32'h0040_0098: Instruction = 32'h2001_0001;
32'h0040_009c: Instruction = 32'h102d_0036;
32'h0040_00a0: Instruction = 32'h2001_0002;
32'h0040_00a4: Instruction = 32'h102d_0032;
32'h0040_00a8: Instruction = 32'h2001_0003;
32'h0040_00ac: Instruction = 32'h102d_002e;
32'h0040_00b0: Instruction = 32'h2001_0004;
32'h0040_00b4: Instruction = 32'h102d_002a;
32'h0040_00b8: Instruction = 32'h2001_0005;
32'h0040_00bc: Instruction = 32'h102d_0026;
32'h0040_00c0: Instruction = 32'h2001_0006;
32'h0040_00c4: Instruction = 32'h102d_0022;
32'h0040_00c8: Instruction = 32'h2001_0007;
32'h0040_00cc: Instruction = 32'h102d_001e;
32'h0040_00d0: Instruction = 32'h2001_0008;
32'h0040_00d4: Instruction = 32'h102d_001a;
32'h0040_00d8: Instruction = 32'h2001_0009;
32'h0040_00dc: Instruction = 32'h102d_0016;
32'h0040_00e0: Instruction = 32'h2001_000a;
32'h0040_00e4: Instruction = 32'h102d_0012;
32'h0040_00e8: Instruction = 32'h2001_000b;
32'h0040_00ec: Instruction = 32'h102d_000e;
32'h0040_00f0: Instruction = 32'h2001_000c;
32'h0040_00f4: Instruction = 32'h102d_000a;
32'h0040_00f8: Instruction = 32'h2001_000d;
32'h0040_00fc: Instruction = 32'h102d_0006;
32'h0040_0100: Instruction = 32'h2001_000e;
32'h0040_0104: Instruction = 32'h102d_0002;
32'h0040_0108: Instruction = 32'h240d_0071;
32'h0040_010c: Instruction = 32'h0810_0061;
32'h0040_0110: Instruction = 32'h240d_0079;
32'h0040_0114: Instruction = 32'h0810_0061;
32'h0040_0118: Instruction = 32'h240d_00bf;
32'h0040_011c: Instruction = 32'h0810_0061;
32'h0040_0120: Instruction = 32'h240d_0039;
32'h0040_0124: Instruction = 32'h0810_0061;
32'h0040_0128: Instruction = 32'h240d_00ff;
32'h0040_012c: Instruction = 32'h0810_0061;
32'h0040_0130: Instruction = 32'h240d_0077;
```

```
32'h0040_0134: Instruction = 32'h0810_0061;
32'h0040_0138: Instruction = 32'h240d_006f;
32'h0040_013c: Instruction = 32'h0810_0061;
32'h0040_0140: Instruction = 32'h240d_007f;
32'h0040_0144: Instruction = 32'h0810_0061;
32'h0040_0148: Instruction = 32'h240d_0007;
32'h0040_014c: Instruction = 32'h0810_0061;
32'h0040_0150: Instruction = 32'h240d_007d;
32'h0040_0154: Instruction = 32'h0810_0061;
32'h0040_0158: Instruction = 32'h240d_006d;
32'h0040_015c: Instruction = 32'h0810_0061;
32'h0040_0160: Instruction = 32'h240d_0066;
32'h0040_0164: Instruction = 32'h0810_0061;
32'h0040_0168: Instruction = 32'h240d_004f;
32'h0040_016c: Instruction = 32'h0810_0061;
32'h0040_0170: Instruction = 32'h240d_005b;
32'h0040_0174: Instruction = 32'h0810_0061;
32'h0040_0178: Instruction = 32'h240d_0006;
32'h0040_017c: Instruction = 32'h0810_0061;
32'h0040_0180: Instruction = 32'h240d_003f;
32'h0040_0184: Instruction = 32'h018d_6025;
32'h0040_0188: Instruction = 32'h3c01_4000;
32'h0040_018c: Instruction = 32'h0020_0821;
32'h0040_0190: Instruction = 32'hac2c_0010;
32'h0040_0194: Instruction = 32'h2941_0c00;
32'h0040_0198: Instruction = 32'h1020_0002;
32'h0040_019c: Instruction = 32'h214a_0001;
32'h0040_01a0: Instruction = 32'h0810_0065;
32'h0040_01a4: Instruction = 32'h2129_0001;
32'h0040_01a8: Instruction = 32'h0810_000b;
32'h0040_01ac: Instruction = 32'h2108_0001;
32'h0040_01b0: Instruction = 32'h0810_0004;
32'h0040_01b4: Instruction = 32'h2408_0000;
32'h0040_01b8: Instruction = 32'h8d10_0000;
32'h0040_01bc: Instruction = 32'h8d11_0004;
32'h0040_01c0: Instruction = 32'h8d12_0008;
32'h0040_01c4: Instruction = 32'h8d13_000c;
32'h0040_01c8: Instruction = 32'h2114_0010;
32'h0040_01cc: Instruction = 32'h0013_a880;
32'h0040_01d0: Instruction = 32'h02b4_a820;
32'h0040_01d4: Instruction = 32'h0013_b0c0;
```

```
32'h0040_01d8: Instruction = 32'h02d4_b020;
32'h0040_01dc: Instruction = 32'h2217_0001;
32'h0040_01e0: Instruction = 32'h0017_b880;
32'h0040_01e4: Instruction = 32'h02f6_b820;
32'h0040_01e8: Instruction = 32'h2408_0000;
32'h0040_01ec: Instruction = 32'h7212_1802;
32'h0040_01f0: Instruction = 32'h0103_082a;
32'h0040_01f4: Instruction = 32'h1020_0007;
32'h0040_01f8: Instruction = 32'h2409_0100;
32'h0040_01fc: Instruction = 32'h2001_0004;
32'h0040_0200: Instruction = 32'h7101_5002;
32'h0040_0204: Instruction = 32'h012a_4820;
32'h0040_0208: Instruction = 32'had20_0000;
32'h0040_020c: Instruction = 32'h2108_0001;
32'h0040_0210: Instruction = 32'h0810_007c;
32'h0040_0214: Instruction = 32'h240c_0000;
32'h0040_0218: Instruction = 32'h0190_082a;
32'h0040_021c: Instruction = 32'h1020_0029;
32'h0040_0220: Instruction = 32'h2001_0004;
32'h0040_0224: Instruction = 32'h7181_c002;
32'h0040_0228: Instruction = 32'h02d8_c020;
32'h0040_022c: Instruction = 32'h8f0b_0000;
32'h0040_0230: Instruction = 32'h8f18_0004;
32'h0040_0234: Instruction = 32'h000b_6820;
32'h0040_0238: Instruction = 32'h01b8_082a;
32'h0040_023c: Instruction = 32'h1020_001f;
32'h0040_0240: Instruction = 32'h2001_0004;
32'h0040_0244: Instruction = 32'h71a1_7002;
32'h0040_0248: Instruction = 32'h02ae_7020;
32'h0040_024c: Instruction = 32'h8dce_0000;
32'h0040_0250: Instruction = 32'h2001_0004;
32'h0040_0254: Instruction = 32'h71a1_c802;
32'h0040_0258: Instruction = 32'h0299_c820;
32'h0040_025c: Instruction = 32'h8f39_0000;
32'h0040_0260: Instruction = 32'h240f_0000;
32'h0040_0264: Instruction = 32'h01f2_082a;
32'h0040_0268: Instruction = 32'h1020_0012;
32'h0040_026c: Instruction = 32'h7192_4802;
32'h0040_0270: Instruction = 32'h012f_4820;
32'h0040_0274: Instruction = 32'h2001_0004;
32'h0040_0278: Instruction = 32'h7121_4802;
```

```
32'h0040_027c: Instruction = 32'h240b_0100;
32'h0040_0280: Instruction = 32'h012b_4820;
32'h0040_0284: Instruction = 32'h71d2_5002;
32'h0040_0288: Instruction = 32'h014f_5020;
32'h0040_028c: Instruction = 32'h2001_0004;
32'h0040_0290: Instruction = 32'h7141_5002;
32'h0040_0294: Instruction = 32'h0157_5020;
32'h0040_0298: Instruction = 32'h8d2b_0000;
32'h0040_029c: Instruction = 32'h8d4a_0000;
32'h0040_02a0: Instruction = 32'h7159_5002;
32'h0040_02a4: Instruction = 32'h016a_5820;
32'h0040_02a8: Instruction = 32'had2b_0000;
32'h0040_02ac: Instruction = 32'h21ef_0001;
32'h0040_02b0: Instruction = 32'h0810_0099;
32'h0040_02b4: Instruction = 32'h21ad_0001;
32'h0040_02b8: Instruction = 32'h0810_008e;
32'h0040_02bc: Instruction = 32'h218c_0001;
32'h0040_02c0: Instruction = 32'h0810_0086;
32'h0040_02c4: Instruction = 32'h7212_1002;
32'h0040_02c8: Instruction = 32'h03e0_0008;
32'h0040_02cc: Instruction = 32'h2403_0000;
32'h0040_02d0: Instruction = 32'h2418_ffff;
32'h0040_02d4: Instruction = 32'h0018_c180;
32'h0040_02d8: Instruction = 32'h0318_c027;
32'h0040_02dc: Instruction = 32'h0003_c982;
32'h0040_02e0: Instruction = 32'h0338_c824;
32'h0040_02e4: Instruction = 32'h0019_c902;
32'h0040_02e8: Instruction = 32'h2001_0001;
32'h0040_02ec: Instruction = 32'h1039_0009;
32'h0040_02f0: Instruction = 32'h2001_0002;
32'h0040_02f4: Instruction = 32'h1039_000c;
32'h0040_02f8: Instruction = 32'h2001_0003;
32'h0040_02fc: Instruction = 32'h1039_000f;
32'h0040_0300: Instruction = 32'h240c_0279;
32'h0040_0304: Instruction = 32'h3c01_4000;
32'h0040_0308: Instruction = 32'h0020_0821;
32'h0040_030c: Instruction = 32'hac2c_0010;
32'h0040_0310: Instruction = 32'h0810_00d4;
32'h0040_0314: Instruction = 32'h240c_0454;
32'h0040_0318: Instruction = 32'h3c01_4000;
32'h0040_031c: Instruction = 32'h0020_0821;
```

```
32'h0040_0320: Instruction = 32'hac2c_0010;
32'h0040_0324: Instruction = 32'h0810_00d4;
32'h0040_0328: Instruction = 32'h240c_085e;
32'h0040_032c: Instruction = 32'h3c01_4000;
32'h0040_0330: Instruction = 32'h0020_0821;
32'h0040_0334: Instruction = 32'hac2c_0010;
32'h0040_0338: Instruction = 32'h0810_00d4;
32'h0040_033c: Instruction = 32'h240c_0000;
32'h0040_0340: Instruction = 32'h3c01_4000;
32'h0040_0344: Instruction = 32'h0020_0821;
32'h0040_0348: Instruction = 32'hac2c_0010;
32'h0040_034c: Instruction = 32'h0810_00d4;
32'h0040_0350: Instruction = 32'h2063_0001;
32'h0040_0354: Instruction = 32'h0810_00b7;
32'h0040_0358: Instruction = 32'h0000_0000;
32'h0040_035c: Instruction = 32'h0000_0000;
32'h0040_0360: Instruction = 32'h0000_0000;
32'h0040_0364: Instruction = 32'h0000_0000;
32'h0040_0368: Instruction = 32'h0000_0000;
32'h0040_036c: Instruction = 32'h0000_0000;
32'h0040_0370: Instruction = 32'h0000_0000;
32'h0040_0374: Instruction = 32'h0000_0000;
32'h0040_0378: Instruction = 32'h0000_0000;
32'h0040_037c: Instruction = 32'h0000_0000;
32'h0040_0380: Instruction = 32'h0000_0000;
32'h0040_0384: Instruction = 32'h0000_0000;
32'h0040_0388: Instruction = 32'h0000_0000;
32'h0040_038c: Instruction = 32'h0000_0000;
32'h0040_0390: Instruction = 32'h0000_0000;
32'h0040_0394: Instruction = 32'h0000_0000;
32'h0040_0398: Instruction = 32'h0000_0000;
32'h0040_039c: Instruction = 32'h0000_0000;
32'h0040_03a0: Instruction = 32'h0000_0000;
32'h0040_03a4: Instruction = 32'h0000_0000;
32'h0040_03a8: Instruction = 32'h0000_0000;
32'h0040_03ac: Instruction = 32'h0000_0000;
32'h0040_03b0: Instruction = 32'h0000_0000;
32'h0040_03b4: Instruction = 32'h0000_0000;
32'h0040_03b8: Instruction = 32'h0000_0000;
32'h0040_03bc: Instruction = 32'h0000_0000;
32'h0040_03c0: Instruction = 32'h0000_0000;
```

```
32'h0040_03c4: Instruction = 32'h0000_0000;
32'h0040_03c8: Instruction = 32'h0000_0000;
32'h0040_03cc: Instruction = 32'h0000_0000;
32'h0040_03d0: Instruction = 32'h0000_0000;
32'h0040_03d4: Instruction = 32'h0000_0000;
32'h0040_03d8: Instruction = 32'h0000_0000;
32'h0040_03dc: Instruction = 32'h0000_0000;
32'h0040_03e0: Instruction = 32'h0000_0000;
32'h0040_03e4: Instruction = 32'h0000_0000;
32'h0040_03e8: Instruction = 32'h0000_0000;
32'h0040_03ec: Instruction = 32'h0000_0000;
32'h0040_03f0: Instruction = 32'h0000_0000;
32'h0040_03f4: Instruction = 32'h0000_0000;
32'h0040_03f8: Instruction = 32'h0000_0000;
32'h0040_03fc: Instruction = 32'h0000_0000;
default:       Instruction = 32'h0000_0000;
```

A.3 仿真代码

Listing 12: user/sim/asm2inst.py

```python
import os
import argparse

def asm2inst(asm_file, inst_file, mode='HexText'):
    """
    Convert assembly instructions to machine code instructions.

    :param asm_file: Path to the input assembly file.
    :param inst_file: Path to the output instruction file.
    """
    if not os.path.exists(asm_file):
        raise FileNotFoundError(f"Assembly file '{asm_file}' does not exist.")

    cmd = f"java -jar Mars4_5.jar {asm_file} a dump .text HexText {inst_file}"
    os.system(cmd)
    lines = []
    with open(inst_file, 'r', encoding='utf-8') as f:
        lines = f.readlines()
    with open(inst_file, 'w', encoding='utf-8') as f:
        f.writelines(lines + ["00000000\n"] * (256 - len(lines)))
```

```

    if mode == 'VerilogCase':
        new_lines = []
        with open(inst_file, 'r', encoding='utf-8') as f:
            lines = f.readlines()
        inst_idx = 0
        for line in lines:
            new_lines.append(f"32'h0040_{inst_idx:04x}: Instruction = 32'h{line[:4]}_{line[4:8]};\n")
            inst_idx += 4
        with open(inst_file, 'w', encoding='utf-8') as f:
            f.writelines(new_lines)
            f.write("default:       Instruction = 32'h0000_0000;\n")

if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Convert assembly instructions to machine code instructions.")
    parser.add_argument("asm_file", help="Path to the input assembly file.")
    parser.add_argument("inst_file", help="Path to the output instruction file.")
    parser.add_argument("--mode", "-m", help="Convert mode (HexText or VerilogCase)", default=None)
    args = parser.parse_args()
    if not os.path.exists(args.asm_file):
        print(f"Error: Assembly file '{asm_file}' does not exist.")
        exit(1)

    try:
        asm2inst(args.asm_file, args.inst_file, args.mode)
        print(f"Converted '{args.asm_file}' to '{args.inst_file}' successfully.")
    except Exception as e:
        print(f"Error: {e}")
```

Listing 13: user/sim/printbin.py

```
import argparse
import os

def printbin(filename, byteorder='big', outputorder='big'):
    with open(filename, 'rb') as f:
        content = f.read()
    for i in range(0, len(content), 4):
```

```
        if i + 4 <= len(content):
            word = content[i: i+4]
            word = int.from_bytes(word, byteorder=byteorder)
            word = word.to_bytes(4, byteorder=outputorder)
            # 32'h format
            print(f"memory[{i//4:2d}] <= 32'h{word[0]:02x}{word[1]:02x}_{word[2]:02x}{word[3]:02x};")
        else:
            # Handle the case where the last word is less than 4 bytes
            word = content[i:] + b'\x00' * (4 - len(content[i:]))
            word = int.from_bytes(word, byteorder=byteorder)
            word = word.to_bytes(4, byteorder=outputorder)
            print(f"memory[{i//4:2d}] <= 32'h{word[0]:02x}{word[1]:02x}_{word[2]:02x}{word[3]:02x};")


if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Print binary representation of a file.")
    parser.add_argument("filename", help="Path to the binary file.")
    parser.add_argument("--byteorder", "-bo", choices=['big', 'little'], default='big',
                        help="Byte order of the input file (default: big).")
    parser.add_argument("--outputorder", "-oo", choices=['big', 'little'], default='big',
                        help="Byte order of the output (default: big).")
    args = parser.parse_args()
    if not os.path.exists(args.filename):
        print(f"Error: File '{args.filename}' does not exist.")
        exit(1)

    try:
        printbin(args.filename, byteorder=args.byteorder, outputorder=args.outputorder)
    except Exception as e:
        print(f"Error: {e}")
        exit(1)
```

Listing 14: user/sim/sparse_matmul_mars.asm

```
# This file is used to get the instruction count of the sparse matrix multiplication function in MARS simulator.
.data
    mat:    .word   0x00000003, 0x00000004, 0x00000005, 0x00000004,
                    0x00000009, 0x00000007, 0x0000000f, 0x00000009,
```

```
5                   0x00000002, 0x00000001, 0x00000000, 0x00000002,
6                   0x00000000, 0x00000001, 0x00000002, 0x00000004,
7                   0x00000001, 0x00000004, 0x00000000, 0x0000000c,
8                   0x0000000b, 0x00000009, 0x00000000, 0x0000000b,
9                   0x00000008, 0x00000002, 0x0000000c, 0x00000002,
10                  0x0000000b, 0x0000000a, 0x00000000, 0x0000000a,
11                  0x0000000c, 0x00000000, 0x00000001, 0x00000009
12
13  .text
14      la      $t0,    0x00000000
15      lw      $s0,    0x00000000($t0)             # $s0: m
16      lw      $s1,    0x00000004($t0)             # $s1: n
17      lw      $s2,    0x00000008($t0)             # $s2: p
18      lw      $s3,    0x0000000c($t0)             # $s3: s
19
20      addi    $s4,    $t0,            16          # $s4: values
21
22      sll     $s5,    $s3,            2
23      add     $s5,    $s5,            $s4         # $s5: col_indices
24
25      sll     $s6,    $s3,            3
26      add     $s6,    $s6,            $s4         # $s6: row_ptr
27
28      addi    $s7,    $s0,            1
29      sll     $s7,    $s7,            2
30      add     $s7,    $s7,            $s6         # $s7: B
31      # set zeros
32      li      $t0,    0                           # int i = 0;
33      mul     $v1,    $s0,            $s2         # $v1: m * p
34      loop_0  :
35      bge     $t0,    $v1,            end_loop_0  # if (i < m * p) next, otherwise to
    end_loop_0
36
37      la      $t1,    0x00000100                  # $t1: C
38      mul     $t2,    $t0,            4           # $t2: (int*)%i;
39      add     $t1,    $t1,            $t2         # $t1 = C + i;
40      sw      $zero,  0x00000000($t1)             # C[i] = 0;
41
42      addi    $t0,    $t0,            1           # i++;
43      j       loop_0                              # go back to the beginning of the loop
44  end_loop_0:
```

```
45
46      # the big loop, with $t3 used multiple times in different loops
47      # t3: start, t4: i, t5: j, t6: k, t7: l, t8: end, t9: val
48      li      $t4,    0                               # int i = 0;
49      loop_3  :
50      bge     $t4,    $s0,                end_loop_3  # if (i < m) next, otherwise to
    end_loop_3
51
52      mul     $t8,    $t4,                4           # end = (int*)i;
53      add     $t8,    $s6,                $t8         # end = row_ptr[i];
54      lw      $t3,    0x00000000($t8)                 # start = row_ptr[i];
55      lw      $t8,    0x00000004($t8)                 # end = row_ptr[i + 1]
56
57      # middle loop
58      add     $t5,    $zero,              $t3         # int j = start;
59      loop_2  :
60      bge     $t5,    $t8,                end_loop_2  # if (j < end) next, otherwise to
    end_loop_2
61
62      mul     $t6,    $t5,                4           # k = (int*)j;
63      add     $t6,    $s5,                $t6         # k = &col_indices[j];
64      lw      $t6,    0x00000000($t6)                 # k = col_indices[j];
65
66      mul     $t9,    $t5,                4           # val = (int*)j;
67      add     $t9,    $s4,                $t9         # val = &values[j];
68      lw      $t9,    0x00000000($t9)                 # val = values[j];
69
70      # inner loop
71      li      $t7,    0                               # int l = 0;
72      loop_1  :
73      bge     $t7,    $s2,                end_loop_1  # if (l < p) next, otherwise to
    end_loop_1
74
75      mul     $t1,    $t4,                $s2         # $t1: i * p;
76      add     $t1,    $t1,                $t7         # $t1 = i * p + l;
77      mul     $t1,    $t1,                4           # (int*)$t1;
78      la      $t3,    0x00000100                      # $t3: C
79      add     $t1,    $t1,                $t3         # $t1 = &C[i * p + l];
80
81      mul     $t2,    $t6,                $s2         # $t2: k * p;
82      add     $t2,    $t2,                $t7         # $t2 = k * p + l;
```

```
83      mul     $t2,    $t2,            4           # (int*)$t2;
84      add     $t2,    $t2,            $s7         # $t2 = &B[k * p + l];
85
86      lw      $t3,    0x00000000($t1)             # $t3: c_val = C[i * p + l];
87      lw      $t2,    0x00000000($t2)             # $t2 = B[k * p + l];
88      mul     $t2,    $t2,            $t9         # $t2 = val * B[k * p + l];
89      add     $t3,    $t3,            $t2         # c_val = c_val + val * B[k * p + l];
90      sw      $t3,    0x00000000($t1)             # C[i * p + l] = c_val;
91
92      addi    $t7,    $t7,            1           # l++;
93      j       loop_1                              # go back to the beginning of the loop
94  end_loop_1:
95      addi    $t5,    $t5,            1           # j++;
96      j       loop_2                              # go back to the beginning of the loop
97  end_loop_2:
98
99      addi    $t4,    $t4,            1           # i++;
100     j       loop_3                              # go back to the beginning of the loop
101 end_loop_3:
102     mul     $v0,    $s0,            $s2         # calculate m * p;
```

Listing 15: user/sim/tb_sparse_matmul_alog.v

```
1   // Testbench 3 for CPU module
2   // Use iverilog to run this testbench
3   // Tested sparse_matmul process
4
5   `timescale 1ns / 1ps
6   `define SIMULATION
7
8   module tb_sparse_matmul_alog ();
9       reg             clk;
10      reg             reset;
11      wire [12 -1: 0] BCD7;
12
13      // Instantiate the CPU module
14      CPU #(
15          .INST_FILE("user/sim/tb_sparse_matmul_alog.inst")
16      ) u_cpu (
17          .clk   (clk),
18          .reset (reset),
```

```
        .BCD7   (BCD7)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // Toggle clock every 5 time units
    end

    // Reset generation
    initial begin
        reset = 1; // Assert reset
        #10 reset = 0; // Deassert reset after 10 time units
    end

    // Simulation control
    initial begin
        #10000; // Run simulation for a certain time
        $finish; // End simulation
    end

    // Save waveform data
    initial begin
        $dumpfile("prj/icarus/tb_sparse_matmul_alog.vcd"); // Dump waveform data
        $dumpvars(0, tb_sparse_matmul_alog); // Dump all variables in the testbench
    end
endmodule
```

Listing 16: user/sim/tb_CPU_inst_1.asm

```
lui     $s0,    0x00ff          # $s0 = 0x00ff_0000
addi    $s1,    $s1,    3       # $s1 = 0x0000_0003
add     $s2,    $s1,    $s0     # $s2 = 0x00ff_0003
addu    $s3,    $s2,    $s1     # $s3 = 0x00ff_0006
sub     $s4,    $s3,    $s2     # $s4 = 0x0000_0003
subu    $s5,    $s4,    $s3     # $s5 = 0xff00_fffd
srl     $s6,    $s5,    6       # $s6 = 0x03fc_03ff
addi    $s7,    $s7,    3       # $s7 = 0x0000_0003
mul     $s0,    $s6,    $s7     # $s0 = 0x0bf4_0bfd
addiu   $s1,    $s0,    0x14    # $s1 = 0x0bf4_0c11
and     $s2,    $s2,    $s1     # $s2 = 0x00f4_0001
```

```
andi    $s3,    $s2,    1       # $s3 = 0x0000_0001
or      $s4,    $s3,    $s1     # $s4 = 0x0bf4_0c11
xor     $s5,    $s4,    $s3     # $s5 = 0x0bf4_0c10
xor     $s6,    $s6,    $s5     # $s6 = 0x0808_0fef
nor     $s7,    $s6,    $s5     # $s7 = 0xf403_f000
andi    $s0,    $s7,    1023    # $s0 = 0x0000_0000
sll     $s1,    $s0,    1       # $s1 = 0x0000_0000
sra     $s2,    $s1,    3       # $s2 = 0x0000_0000
slt     $s3,    $s2,    $s1     # $s3 = 0x0000_0000
slti    $s4,    $s3,    0x10    # $s4 = 0x0000_0001
sltu    $s5,    $s4,    $s3     # $s5 = 0x0000_0000
sltiu   $s6,    $s5,    0x20    # $s6 = 0x0000_0001
ori     $s7,    $s6,    0x0010  # $s7 = 0x0000_0011
lui     $a0,    0x4000          # $a0 = 0x4000_0000
addi    $a0,    $a0,    0x10    # $a0 = 0x4000_0010
sw      $s2,    0($a0)
lw      $s7,    0($a0)
addi    $s7,    $s7,    1
sw      $s7,    0($a0)
sw      $s6,    0($a0)
# s2 = 0x0000_00fa
# s7 = 0x0000_00fb
# s6 = 0x0000_0001
```

Listing 17: user/sim/tb_CPU_inst_1.v

```
// Testbench 1 for CPU module
// Use iverilog to run this testbench
// Tested all R-type (including xxx-i) instructions and lw, sw instructions
// Tested load-use hazard detection and EXMEM -> ALU, MEMWB -> ALU and MEMWB -> DataMEM
forwarding

`timescale 1ns / 1ps
`define SIMULATION
`define NOT_SPARSE_MATMUL

module tb_CPU_inst_1 ();
    reg             clk;
    reg             reset;
    wire [12 -1: 0] BCD7;

```

```
    // Instantiate the CPU module
    CPU #(
        .INST_FILE("user/sim/tb_CPU_inst_1.inst")
    ) u_cpu (
        .clk   (clk),
        .reset (reset),
        .BCD7  (BCD7)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // Toggle clock every 5 time units
    end

    // Reset generation
    initial begin
        reset = 1; // Assert reset
        #10 reset = 0; // Deassert reset after 10 time units
    end

    // Simulation control
    initial begin
        #100000; // Run simulation for a certain time
        $finish; // End simulation
    end

    // Save waveform data
    initial begin
        $dumpfile("prj/icarus/tb_CPU_inst_1.vcd"); // Dump waveform data
        $dumpvars(0, tb_CPU_inst_1); // Dump all variables in the testbench
    end
endmodule
```

Listing 18: user/sim/tb_CPU_inst_2.asm

```
addi        $t0,    $t0,    0x00ff # $t0 = 0x0000_00ff
    addi    $t1,    $t0,    1      # $t1 = 0x0000_0100
    addi    $t2,    $t0,    2      # $t2 = 0x0000_0101
    addi    $t3,    $zero,  -2     # $t3 = -2
    addi    $t4,    $zero,  2      # $t4 = 2
```

```
l1: addi    $t0,    $t0,    1       # $t0 = 0x0000_0100
    beq     $t0,    $t1,    l1      # beq once
    bne     $t0,    $t2,    l1      # no bne
l2: addi    $t3,    $t3,    1       # $t3 = -1
    bltz    $t3,    l2              # bltz once
    blez    $t3,    l2              # blez once
l3: addi    $t4,    $t4,    -1      # $t4 = 1
    bgtz    $t4,    l3              # bgtz once
    j       j1                      # jump to j1
    addi    $s0,    $s0,    0x0fff # no effect, $s0 = 0
j1: jal     j2                      # jump to j2
    addi    $s2,    $s2,    0x0fff # after jr, $s2 = 0x0000_0fff
    j       j3                      # jump to j3
j2: addi    $s1,    $s1,    0x00ff # $s1 = 0x0000_00ff
    jr      $ra                     # return to j2's next instruction
j3: jal     j4                      # jump to j4 (to set $ra for jalr)
    addi    $s4,    $s4,    0x00ff # after jalr, $s4 = 0x0000_00ff
    j       j5                      # jump to j5
j4: add     $v0,    $ra,    $zero   # $v0 = $ra
    jalr    $s3,    $v0             # jump to $s3 == $ra, $v0 = PC+4
j5: jal     j6                      # jump to j6
    j       ed                      # jump to ed
j6: sw      $ra,    0($zero)        # store $ra to 0($zero)
    lw      $v0,    0($zero)        # load $v0 from 0($zero)
    jr      $v0                     # jump to $v0 (test lw-jal)
ed: addi    $s5,    $s5,    0x00ff # end, $s5 = 0x0000_00ff
```

Listing 19: user/sim/tb_CPU_inst_2.v

```
// Testbench 2 for CPU module
// Use iverilog to run this testbench
// Tested all branch instructions and j instructions

`timescale 1ns / 1ps
`define SIMULATION
`define NOT_SPARSE_MATMUL

module tb_CPU_inst_2 ();
    reg             clk;
    reg             reset;
    wire [12 -1: 0] BCD7;
```

```

    // Instantiate the CPU module
    CPU #(
        .INST_FILE("user/sim/tb_CPU_inst_2.inst")
    ) u_cpu (
        .clk   (clk),
        .reset (reset),
        .BCD7  (BCD7)
    );

    // Clock generation
    initial begin
        clk = 0;
        forever #5 clk = ~clk; // Toggle clock every 5 time units
    end

    // Reset generation
    initial begin
        reset = 1; // Assert reset
        #10 reset = 0; // Deassert reset after 10 time units
    end

    // Simulation control
    initial begin
        #100000; // Run simulation for a certain time
        $finish; // End simulation
    end

    // Save waveform data
    initial begin
        $dumpfile("prj/icarus/tb_CPU_inst_2.vcd"); // Dump waveform data
        $dumpvars(0, tb_CPU_inst_2); // Dump all variables in the testbench
    end
endmodule
```

Listing 20: user/sim/tb_CPU_inst_3.asm

```
    # do the function
    li      $v1,            0                               # $v1 = 0: symbol of doing
multiplication
    jal     sparse_matmul                                   # jump to the sparse matrix
```

```
multiplication function

    # print the result by BCD LEDs
    li      $v1,            1                           # $v1 = 1: symbol of doing
printing
    move    $t0,            $zero                       # $t0: outest loop variable
mat_loop:                                               # outest loop, controlling the
sequence of the whole matrix
    bge     $t0,            $v0,            end_program

    mul     $t3,            $t0,            4
    addi    $t3,            $t3,            0x00000100
    lw      $t3,            ($t3)                       # $t3: the very number to be
printed this time
    li      $t1,            0                           # $t1: middle loop variable
time_loop:                                              # middle loop, controlling the
sequence of four positions' enabling
    bge     $t1,            16,             end_time    # taking turns of printing each
 position for 4096 loops
    li      $t2,            0
    sll     $t4,            $t1,            30          # an_i = 1 while other an's =
0, if i = $t4 (mod 4), $t4: lit position
    beq     $t4,            0xc0000000,     an3
    beq     $t4,            0x80000000,     an2
    beq     $t4,            0x40000000,     an1
    # only last two bytes are useful, while first two bytes are always zero and ignored
an0:
    li      $t4,            0x0100
    srl     $t5,            $t3,            12
    j       bcd
an1:
    li      $t4,            0x0200
    srl     $t5,            $t3,            8
    j       bcd
an2:
    li      $t4,            0x0400
    srl     $t5,            $t3,            4
    j       bcd
an3:
    li      $t4,            0x0800
    srl     $t5,            $t3,            0
```

```
38
39  bcd:                                                    # $t5: the number to be shown
40      andi    $t5,            $t5,            0xf
41      beq     $t5,            0x0,            is0
42      beq     $t5,            0x1,            is1
43      beq     $t5,            0x2,            is2
44      beq     $t5,            0x3,            is3
45      beq     $t5,            0x4,            is4
46      beq     $t5,            0x5,            is5
47      beq     $t5,            0x6,            is6
48      beq     $t5,            0x7,            is7
49      beq     $t5,            0x8,            is8
50      beq     $t5,            0x9,            is9
51      beq     $t5,            0xa,            isa
52      beq     $t5,            0xb,            isb
53      beq     $t5,            0xc,            isc
54      beq     $t5,            0xd,            isd
55      beq     $t5,            0xe,            ise
56      # $t5: now to be the lit LED indices
57  isf:
58      li      $t5,            0x71
59      j       light
60  ise:
61      li      $t5,            0x79
62      j       light
63  isd:
64      li      $t5,            0xbf
65      j       light
66  isc:
67      li      $t5,            0x39
68      j       light
69  isb:
70      li      $t5,            0xff
71      j       light
72  isa:
73      li      $t5,            0x77
74      j       light
75  is9:
76      li      $t5,            0x6f
77      j       light
78  is8:
```

```
79      li      $t5,            0x7f
80      j       light
81  is7:
82      li      $t5,            0x07
83      j       light
84  is6:
85      li      $t5,            0x7d
86      j       light
87  is5:
88      li      $t5,            0x6d
89      j       light
90  is4:
91      li      $t5,            0x66
92      j       light
93  is3:
94      li      $t5,            0x4f
95      j       light
96  is2:
97      li      $t5,            0x5b
98      j       light
99  is1:
100     li      $t5,            0x06
101     j       light
102 is0:
103     li      $t5,            0x3f
104
105 light:                                          # $t4: to be the full code to
    be saved in memory, instructing LED lighting
106     or      $t4,            $t4,                $t5
107     sw      $t4,            0x40000010($zero)
108
109 show_loop:                                      # innest loop, waiting with the
     same status
110     bge     $t2,            16,                 end_show    # $t2: innest loop variable
111     addi    $t2,            $t2,                1           # $t2++;
112     j       show_loop
113
114 end_show:
115     addi    $t1,            $t1,                1           # $t1++;
116     j       time_loop
117
```

```
118 end_time:
119     addi    $t0,            $t0,            1           # $t0++;
120     j       mat_loop
121
122     # the main function of sparse matrix multiplication, search in the assembly homework
    for more detail
123 sparse_matmul:
124     la      $t0,            0x00000000
125     lw      $s0,            0x00000000($t0)             # $s0: m
126     lw      $s1,            0x00000004($t0)             # $s1: n
127     lw      $s2,            0x00000008($t0)             # $s2: p
128     lw      $s3,            0x0000000c($t0)             # $s3: s
129
130     addi    $s4,            $t0,            16          # $s4: values
131
132     sll     $s5,            $s3,            2
133     add     $s5,            $s5,            $s4         # $s5: col_indices
134
135     sll     $s6,            $s3,            3
136     add     $s6,            $s6,            $s4         # $s6: row_ptr
137
138     addi    $s7,            $s0,            1
139     sll     $s7,            $s7,            2
140     add     $s7,            $s7,            $s6         # $s7: B
141     # set zeros
142     li      $t0,            0                           # int i = 0;
143     mul     $v1,            $s0,            $s2         # $v1: m * p
144     loop_0  :
145     bge     $t0,            $v1,            end_loop_0  # if (i < m * p) next,
    otherwise to end_loop_0
146
147     la      $t1,            0x00000100                  # $t1: C
148     mul     $t2,            $t0,            4           # $t2: (int*)%i;
149     add     $t1,            $t1,            $t2         # $t1 = C + i;
150     sw      $zero,          0x00000000($t1)             # C[i] = 0;
151
152     addi    $t0,            $t0,            1           # i++;
153     j       loop_0                                      # go back to the beginning of
    the loop
154 end_loop_0:
155
```

```
156     # the big loop, with $t3 used multiple times in different loops
157     # t3: start, t4: i, t5: j, t6: k, t7: l, t8: end, t9: val
158     li      $t4,            0                               # int i = 0;
159     loop_3  :
160     bge     $t4,            $s0,                end_loop_3  # if (i < m) next, otherwise to
    end_loop_3
161
162     mul     $t8,            $t4,                4           # end = (int*)i;
163     add     $t8,            $s6,                $t8         # end = row_ptr[i];
164     lw      $t3,            0x00000000($t8)                 # start = row_ptr[i];
165     lw      $t8,            0x00000004($t8)                 # end = row_ptr[i + 1]
166
167     # middle loop
168     add     $t5,            $zero,              $t3         # int j = start;
169     loop_2  :
170     bge     $t5,            $t8,                end_loop_2  # if (j < end) next, otherwise
    to end_loop_2
171
172     mul     $t6,            $t5,                4           # k = (int*)j;
173     add     $t6,            $s5,                $t6         # k = &col_indices[j];
174     lw      $t6,            0x00000000($t6)                 # k = col_indices[j];
175
176     mul     $t9,            $t5,                4           # val = (int*)j;
177     add     $t9,            $s4,                $t9         # val = &values[j];
178     lw      $t9,            0x00000000($t9)                 # val = values[j];
179
180     #inner loop
181     li      $t7,            0                               # int l = 0;
182     loop_1  :
183     bge     $t7,            $s2,                end_loop_1  # if (l < p) next, otherwise to
    end_loop_1
184
185     mul     $t1,            $t4,                $s2         # $t1: i * p;
186     add     $t1,            $t1,                $t7         # $t1 = i * p + l;
187     mul     $t1,            $t1,                4           # (int*)$t1;
188     la      $t3,            0x00000100                      # $t3: C
189     add     $t1,            $t1,                $t3         # $t1 = &C[i * p + l];
190
191     mul     $t2,            $t6,                $s2         # $t2: k * p;
192     add     $t2,            $t2,                $t7         # $t2 = k * p + l;
193     mul     $t2,            $t2,                4           # (int*)$t2;
```

```
194     add     $t2,            $t2,            $s7         # $t2 = &B[k * p + l];
195
196     lw      $t3,            0x00000000($t1)             # $t3: c_val = C[i * p + l];
197     lw      $t2,            0x00000000($t2)             # $t2 = B[k * p + l];
198     mul     $t2,            $t2,            $t9         # $t2 = val * B[k * p + l];
199     add     $t3,            $t3,            $t2         # c_val = c_val + val * B[k * p
        + l];
200     sw      $t3,            0x00000000($t1)             # C[i * p + l] = c_val;
201
202     addi    $t7,            $t7,            1           # l++;
203     j       loop_1                                      # go back to the beginning of
    the loop
204 end_loop_1:
205     addi    $t5,            $t5,            1           # j++;
206     j       loop_2                                      # go back to the beginning of
    the loop
207 end_loop_2:
208
209     addi    $t4,            $t4,            1           # i++;
210     j       loop_3                                      # go back to the beginning of
    the loop
211 end_loop_3:
212     mul     $v0,            $s0,            $s2         # calculate m * p;
213     jr      $ra                                         # return m * p;
214
215     # some ending animation, printing "_End" forever
216 end_program:
217     li      $v1,            0                           # $v1 = 0: ending, $v1 is also
    the loop variable
218     li      $t8,            -1                          # $t8: getting valid positions
    in $t9
219     sll     $t8,            $t8,            6           # 2^6 * T is the loop cycle
220     nor     $t8,            $t8,            $t8
221 denoument:
222     srl     $t9,            $v1,            6           # $t9: $v1 mod 2^6
223     and     $t9,            $t9,            $t8
224     srl     $t9,            $t9,            4           # divide the loop cycle into
    four identical periods
225     beq     $t9,            1,              end1
226     beq     $t9,            2,              end2
227     beq     $t9,            3,              end3
```

```
end0:
    li      $t4,            0x00000279
    sw      $t4,            0x40000010($zero)           # _E__
    j       end_denoument
end1:
    li      $t4,            0x00000454
    sw      $t4,            0x40000010($zero)           # __n_
    j       end_denoument
end2:
    li      $t4,            0x0000085e
    sw      $t4,            0x40000010($zero)           # ___d
    j       end_denoument
end3:
    li      $t4,            0
    sw      $t4,            0x40000010($zero)           # -___
    j       end_denoument
end_denoument:
    addi    $v1,            $v1,            1           # $v1++;
    j       denoument
```

Listing 21: user/sim/tb_CPU_inst_3.v

```
// Testbench 3 for CPU module
// Use iverilog to run this testbench
// Tested sparse_matmul process

`timescale 1ns / 1ps
`define SIMULATION

module tb_CPU_inst_3 ();
    reg             clk;
    reg             reset;
    wire [12 -1: 0] BCD7;

    // Instantiate the CPU module
    CPU #(
        .INST_FILE("user/sim/tb_CPU_inst_3.inst")
    ) u_cpu (
        .clk   (clk),
        .reset (reset),
        .BCD7  (BCD7)
```

```
20      );
21
22      // Clock generation
23      initial begin
24          clk = 0;
25          forever #5 clk = ~clk; // Toggle clock every 5 time units
26      end
27
28      // Reset generation
29      initial begin
30          reset = 1; // Assert reset
31          #10 reset = 0; // Deassert reset after 10 time units
32      end
33
34      // Simulation control
35      initial begin
36          #1000000; // Run simulation for a certain time
37          $finish; // End simulation
38      end
39
40      // Save waveform data
41      initial begin
42          $dumpfile("prj/icarus/tb_CPU_inst_3.vcd"); // Dump waveform data
43          $dumpvars(0, tb_CPU_inst_3); // Dump all variables in the testbench
44      end
45  endmodule
```

Listing 22: user/sim/tb_sparse_matmul_alog.inst

```
24080000
8d100000
8d110004
8d120008
8d13000c
21140010
0013a880
02b4a820
0013b0c0
02d4b020
22170001
0017b880
02f6b820
24080000
72121802
0103082a
10200007
24090100
20010004
71015002
012a4820
ad200000
21080001
0810000f
240c0000
0190082a
10200029
20010004
7181c002
02d8c020
8f0b0000
8f180004
000b6820
01b8082a
1020001f
20010004
71a17002
02ae7020
8dce0000
20010004
71a1c802
0299c820
8f390000
240f0000
01f2082a
10200012
71924802
012f4820
20010004
71214802
240b0100
012b4820
71d25002
014f5020
20010004
71415002
01575020
8d2b0000
8d4a0000
71595002
016a5820
ad2b0000
21ef0001
0810002c
21ad0001
08100021
218c0001
08100019
72121002
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
```

```
82  00000000
83  00000000
84  00000000
85  00000000
86  00000000
87  00000000
88  00000000
89  00000000
90  00000000
91  00000000
92  00000000
93  00000000
94  00000000
95  00000000
96  00000000
97  00000000
98  00000000
99  00000000
100 00000000
101 00000000
102 00000000
103 00000000
104 00000000
105 00000000
106 00000000
107 00000000
108 00000000
109 00000000
110 00000000
111 00000000
112 00000000
113 00000000
114 00000000
115 00000000
116 00000000
117 00000000
118 00000000
119 00000000
120 00000000
121 00000000
122 00000000
123 00000000
124 00000000
125 00000000
126 00000000
127 00000000
128 00000000
129 00000000
130 00000000
131 00000000
132 00000000
133 00000000
134 00000000
135 00000000
136 00000000
137 00000000
138 00000000
139 00000000
140 00000000
141 00000000
142 00000000
143 00000000
144 00000000
145 00000000
146 00000000
147 00000000
148 00000000
149 00000000
150 00000000
151 00000000
152 00000000
153 00000000
154 00000000
155 00000000
156 00000000
157 00000000
158 00000000
159 00000000
160 00000000
161 00000000
162 00000000
163 00000000
```

```
00000000
00000000
00000000
```

```
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
```

Listing 23: user/sim/tb_CPU_inst_1.inst

```
3c1000ff
22310003
02309020
02519821
0272a022
0293a823
0015b182
22f70003
72d78002
26110014
02519024
32530001
0271a025
0293a826
02d5b026
02d5b827
32f003ff
00108840
001190c3
0251982a
2a740010
0293a82b
2eb60020
36d70010
3c044000
20840010
ac920000
8c970000
22f70001
ac970000
ac960000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000
```

```
00000000
00000000
00000000
```

```
151 00000000
152 00000000
153 00000000
154 00000000
155 00000000
156 00000000
157 00000000
158 00000000
159 00000000
160 00000000
161 00000000
162 00000000
163 00000000
164 00000000
165 00000000
166 00000000
167 00000000
168 00000000
169 00000000
170 00000000
171 00000000
172 00000000
173 00000000
174 00000000
175 00000000
176 00000000
177 00000000
178 00000000
179 00000000
180 00000000
181 00000000
182 00000000
183 00000000
184 00000000
185 00000000
186 00000000
187 00000000
188 00000000
189 00000000
190 00000000
191 00000000
192 00000000
193 00000000
194 00000000
195 00000000
196 00000000
197 00000000
198 00000000
199 00000000
200 00000000
201 00000000
202 00000000
203 00000000
204 00000000
205 00000000
206 00000000
207 00000000
208 00000000
209 00000000
210 00000000
211 00000000
212 00000000
213 00000000
214 00000000
215 00000000
216 00000000
217 00000000
218 00000000
219 00000000
220 00000000
221 00000000
222 00000000
223 00000000
224 00000000
225 00000000
226 00000000
227 00000000
228 00000000
229 00000000
230 00000000
231 00000000
232 00000000
```

```
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
```

Listing 24: user/sim/tb_CPU_inst_2.inst

```
210800ff
21090001
210a0002
200bfffe
200c0002
21080001
1109fffe
150afffd
216b0001
0560fffe
1960fffd
218cffff
1d80fffe
0810000f
22100fff
0c100012
22520fff
08100014
223100ff
03e00008
0c100017
229400ff
08100019
03e01020
00409809
0c10001b
0810001e
ac1f0000
8c020000
00400008
22b500ff
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
```

```
00000000
00000000
00000000
```

```
00000000
00000000
00000000
```

```
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
00000000
```

Listing 25: user/sim/tb_CPU_inst_3.inst

```
24030000
0c10006d
24030001
00004021
0102082a
102000ad
20010004
71015802
216b0100
8d6b0000
24090000
29210010
1020005e
240a0000
00096780
3c01c000
34210000
102c000f
3c018000
34210000
102c0009
3c014000
34210000
102c0003
240c0100
000b6b02
08100023
240c0200
000b6a02
08100023
240c0400
000b6902
08100023
240c0800
000b6802
31ad000f
20010000
102d003a
20010001
102d0036
20010002
102d0032
```

```
20010003
102d002e
20010004
102d002a
20010005
102d0026
20010006
102d0022
20010007
102d001e
20010008
102d001a
20010009
102d0016
2001000a
102d0012
2001000b
102d000e
2001000c
102d000a
2001000d
102d0006
2001000e
102d0002
240d0071
08100061
240d0079
08100061
240d00bf
08100061
240d0039
08100061
240d00ff
08100061
240d0077
08100061
240d006f
08100061
240d007f
08100061
240d0007
08100061
240d007d
08100061
240d006d
08100061
240d0066
08100061
240d004f
08100061
240d005b
08100061
240d0006
08100061
240d003f
018d6025
3c014000
00200821
ac2c0010
29410010
10200002
214a0001
08100065
21290001
0810000b
21080001
08100004
24080000
8d100000
8d110004
8d120008
8d13000c
21140010
0013a880
02b4a820
0013b0c0
02d4b020
22170001
0017b880
02f6b820
24080000
72121802
```

```
0103082a
10200007
24090100
20010004
71015002
012a4820
ad200000
21080001
0810007c
240c0000
0190082a
10200029
20010004
7181c002
02d8c020
8f0b0000
8f180004
000b6820
01b8082a
1020001f
20010004
71a17002
02ae7020
8dce0000
20010004
71a1c802
0299c820
8f390000
240f0000
01f2082a
10200012
71924802
012f4820
20010004
71214802
240b0100
012b4820
71d25002
014f5020
20010004
71415002
01575020
8d2b0000
8d4a0000
71595002
016a5820
ad2b0000
21ef0001
08100099
21ad0001
0810008e
218c0001
08100086
72121002
03e00008
24030000
2418ffff
0018c180
0318c027
0003c982
0338c824
0019c902
20010001
10390009
20010002
1039000c
20010003
1039000f
240c0279
3c014000
00200821
ac2c0010
081000d4
240c0454
3c014000
00200821
ac2c0010
081000d4
240c085e
3c014000
00200821
ac2c0010
```

```
081000d4
240c0000
3c014000
00200821
ac2c0010
081000d4
20630001
081000b7
00000000
00000000
00000000
```

A.4 约束文件

Listing 26: user/data/MIPSCPU.xdc

```
# STA
# create_clock -period 16.000 -name sta_clk -waveform {0.000 8.000} [get_ports {clk_in}]

# 100MHz clock
set_property -dict {PACKAGE_PIN R4 IOSTANDARD LVCMOS33} [get_ports {clk_in}]

# reset: KEY3
set_property -dict {PACKAGE_PIN K22 IOSTANDARD LVCMOS33} [get_ports {reset}]

# BCD7
set_property -dict {PACKAGE_PIN N2 IOSTANDARD LVCMOS33} [get_ports {BCD7[0]}]
set_property -dict {PACKAGE_PIN P5 IOSTANDARD LVCMOS33} [get_ports {BCD7[1]}]
set_property -dict {PACKAGE_PIN V5 IOSTANDARD LVCMOS33} [get_ports {BCD7[2]}]
set_property -dict {PACKAGE_PIN U5 IOSTANDARD LVCMOS33} [get_ports {BCD7[3]}]
set_property -dict {PACKAGE_PIN T5 IOSTANDARD LVCMOS33} [get_ports {BCD7[4]}]
set_property -dict {PACKAGE_PIN P1 IOSTANDARD LVCMOS33} [get_ports {BCD7[5]}]
set_property -dict {PACKAGE_PIN W4 IOSTANDARD LVCMOS33} [get_ports {BCD7[6]}]
set_property -dict {PACKAGE_PIN V3 IOSTANDARD LVCMOS33} [get_ports {BCD7[7]}]
set_property -dict {PACKAGE_PIN M2 IOSTANDARD LVCMOS33} [get_ports {BCD7[8]}]
set_property -dict {PACKAGE_PIN P2 IOSTANDARD LVCMOS33} [get_ports {BCD7[9]}]
set_property -dict {PACKAGE_PIN R1 IOSTANDARD LVCMOS33} [get_ports {BCD7[10]}]
set_property -dict {PACKAGE_PIN Y3 IOSTANDARD LVCMOS33} [get_ports {BCD7[11]}]
```